Санкт-Петербургский государственный университет

Горошко Андрей Витальевич

Выпускная квалификационная работа

Анализ влияния дообучения YandexGPT на корректность выбора функции для получения данных по исходному запросу

Уровень образования: бакалавриат

Направление *02.03.01 «Математика и компьютерные науки»*

Основная образовательная программа *СВ.5001.2022 «Математика и компьютерные науки»*

Научный руководитель:
доцент кафедры информатики, Григорьева А. В.

Рецензент:
ст. преп. кафедры компьютерных технологий и систем, к. ф.-м. н., Севостьянов Р. А.

Санкт-Петербург
2026

Оглавление

1. Введение **4**

2. Постановка задачи **7**

2.1. Формальная постановка 7

2.2. Гипотезы исследования 8

2.3. Исследовательские вопросы 9

2.4. Критерии успеха 9

2.5. Ограничения исследования 10

3. Обзор литературы **12**

3.1. Методы дообучения больших языковых моделей 12

3.2. Гипотеза поверхностного выравнивания 13

3.3. AI-агенты и выбор инструментов 14

3.4. Синтетическая генерация обучающих данных 16

3.5. Выводы обзора 17

4. Математические основы **18**

4.1. Формализация задачи бинарной классификации 18

4.2. Метрики качества классификации 19

4.3. Низкоранговая адаптация (LoRA) 20

4.4. Статистические критерии сравнения классификаторов . 21

4.5. Бутстрап доверительные интервалы 25

5. Методология экспериментов **27**

5.1. Пайплайн дообучения и оценки 27

5.2. Описание бенчмарков 28

5.3. Обучающие датасеты 31

5.4. Протокол дообучения 33

5.5. Дизайн эксперимента 34

6. Результаты экспериментов и их анализ **37**

6.1. Оценка базовой модели 37

6.2. Основные результаты дообучения 38

6.3. Валидация на реальных данных 39
6.4. Анализ кривой обучения 41
6.5. Анализ типа обучающих данных 43
6.6. Анализ ошибок 44
6.7. Статистическая значимость 49
6.8. Ответы на исследовательские вопросы 51

7. Угрозы валидности 53
7.1. Внутренняя валидность 53
7.2. Внешняя валидность 54
7.3. Конструктная валидность 54
7.4. Статистическая валидность 55

8. Программная реализация 56
8.1. Архитектура системы 56
8.2. Подсистема статистического анализа 57
8.3. Подсистема инференса 58
8.4. Подсистема генерации данных 59
8.5. Обеспечение качества кода 59
8.6. Технологический стек 59

9. Заключение 61
9.1. Направления дальнейших исследований 63

Список литературы 65

1. Введение

Одним из центральных направлений развития систем на основе больших языковых моделей (LLM) является создание автономных агентов, способных взаимодействовать с внешними инструментами — API, базами данных, поисковыми системами — для решения задач, выходящих за пределы генерации текста. Подход, при котором модель самостоятельно определяет, какой инструмент вызвать и с какими параметрами, получил название *tool use* (или *function calling*) и активно исследуется в работах [30, 14, 28]. Качество выбора инструмента непосредственно определяет применимость агента в промышленных системах: ошибка классификации приводит либо к некорректному ответу, либо к полному отсутствию результата.

В настоящей работе рассматривается AI-агент в сфере недвижимости, обрабатывающий пользовательские запросы на естественном языке. Агент должен классифицировать каждый запрос в один из двух классов и вызвать соответствующую функцию: `ResolveEntities` для географических объектов (класс GEO) и `SearchTags` для характеристик объектов недвижимости (класс TAG). Базовая модель YandexGPT 5.1 Pro [32], используемая в качестве ядра агента, на объединённом бенчмарке из 500 примеров достигает точности (accuracy) лишь 81,8% при Macro $F_1 = 0{,}526$. Анализ ошибок выявляет систематический сдвиг (bias): из 174 запросов класса TAG верно классифицированы лишь 49,4%, а для 34 запросов модель вообще не генерирует вызов инструмента. Таким образом, базовая модель обладает выраженным предпочтением класса GEO, что делает её непригодной для промышленного применения без дополнительной адаптации.

Центральная рабочая гипотеза данного исследования состоит в том, что выбор инструмента может быть интерпретирован как устойчивый *паттерн формы ответа* модели — по аналогии с гипотезой поверхностного выравнивания (Superficial Alignment Hypothesis) Zhou и др. [16, 12], согласно которой дообучение с инструкциями преимущественно коррек-

тирует формат и стиль генерации, а не фактические знания модели (подробно — раздел 3). Настоящая работа проверяет применимость этой аналогии к задаче tool selection: если она верна, то даже небольшой объём обучающих примеров, дообученных методом LoRA [19], должен радикально улучшить качество классификации.

Целью работы является экспериментальная проверка указанной гипотезы. Для этого решаются следующие задачи:

1. формализация задачи классификации инструментов как задачи бинарной классификации с двумя бенчмарками (GEO и TAG);

2. разработка инфраструктуры для дообучения и систематической оценки моделей;

3. оценка 12 конфигураций моделей (1 базовая и 11 дообученных) с вариацией типа обучающих данных (GEO, TAG, смешанные) и объёма выборки;

4. статистический анализ результатов с применением критериев Кохрена Q и Макнемара для попарных сравнений моделей.

Научная новизна работы заключается в исследовании применения метода LoRA для коммерческой LLM на специфичном русскоязычном гео-домене с использованием строгих статистических критериев оценки. В отличие от существующих работ, сосредоточенных преимущественно на англоязычных моделях открытого доступа, данное исследование включает строгое статистическое сравнение 12 конфигураций моделей на двух независимых бенчмарках. Основной результат — достижение точности 99,4% лучшей дообученной моделью при всего одной ошибке на 500 примерах — согласуется с рабочей гипотезой о стилистической природе выбора инструмента и демонстрирует практическую эффективность подхода.

Работа организована следующим образом. В разделе 2 дана формальная постановка задачи. Раздел 3 содержит обзор литературы по tool use, методам дообучения и гипотезе поверхностного выравнивания.

В разделе 4 описаны теоретические основы используемых методов. Раздел 5 посвящён методологии экспериментов, раздел 6 — анализу результатов. Раздел 7 обсуждает угрозы валидности полученных результатов. В разделе 8 описана программная реализация. Раздел 9 содержит заключение и направления дальнейших исследований.

2. Постановка задачи

2.1. Формальная постановка

Пусть $\mathcal{X}$ — пространство пользовательских запросов на естественном языке, поступающих AI-агенту в предметной области поиска объектов недвижимости. Определим конечное множество инструментов (tools):

$$\mathcal{T} = \{t_{\text{GEO}},\ t_{\text{TAG}}\},$$

где $t_{\text{GEO}} \equiv$ `ResolveEntities` — инструмент разрешения географических сущностей (город, район, улица, адрес), а $t_{\text{TAG}} \equiv$ `SearchTags` — инструмент поиска по характеристикам объекта (площадь, тип ремонта, этажность, количество комнат).

Определение 1. *Классификатором первого вызова инструмента называется отображение*

$$f_\theta \colon \mathcal{X} \to \mathcal{T},$$

реализуемое большой языковой моделью (LLM) с параметрами θ, которое каждому входному запросу $x \in \mathcal{X}$ ставит в соответствие инструмент $t \in \mathcal{T}$, выбираемый моделью в качестве первого вызова инструмента (`tool_call`) в генерируемом ответе.

В качестве базовой линии (baseline) для сравнения используется модель YandexGPT 5.1 Pro (f_{θ_0}), не подвергавшаяся дообучению. Дообученные модели f_{θ^*} получаются из YandexGPT Lite — компактной версии семейства YandexGPT — методом LoRA (Low-Rank Adaptation) через Yandex Cloud API.

Введём ошибки классификации в терминах проверки статистических гипотез. Примем за нулевую гипотезу H_0: «запрос x относится к классу TAG».

Определение 2. Ошибка I рода*: запрос класса TAG классифицирован как GEO, то есть $f_\theta(x) = t_{\text{GEO}}$ при истинной метке $y(x) = t_{\text{TAG}}$. Такая ошибка приводит к тому, что агент пытается найти несу-*

ществующий географический объект вместо применения фильтра по характеристикам.

Определение 3. Ошибка II рода*: запрос класса GEO классифицирован как TAG, то есть $f_\theta(x) = t_{\text{TAG}}$ при истинной метке $y(x) = t_{\text{GEO}}$. Такая ошибка приводит к потере географической привязки в результатах поиска.*

Дополнительно выделяется третий исход: модель не генерирует вызов инструмента вовсе, что является критическим сбоем в контексте AI-агента. Обозначим долю таких ответов как $\rho_\varnothing$.

2.2. Гипотезы исследования

Гипотеза 1. *(H1a, значимость улучшения.) Дообучение модели YandexGPT Lite методом LoRA статистически значимо снижает долю ошибок классификации первого вызова инструмента на бенчмарке* `benchmark-combined` *по сравнению с базовой моделью YandexGPT 5.1 Pro (accuracy = 81,8%). Проверка — односторонний точный критерий Макнемара, $\alpha = 0{,}05$.*

Гипотеза 2. *(H1b, превышение порога.) Нижняя граница 95%-го bootstrap-доверительного интервала accuracy лучшей дообученной модели на бенчмарке* `benchmark-combined` *превышает 0,90, что соответствует требованию прикладной системы к точности классификации.*

Гипотеза 3. *Качество обучающих данных (сбалансированный микс реальных пользовательских запросов и синтетических примеров) оказывает большее влияние на итоговое качество классификации, чем объём обучающей выборки. В частности, модель, обученная на меньшем, но качественно подготовленном датасете, превосходит модель, обученную на большем, но несбалансированном датасете.*

2.3. Исследовательские вопросы

Для систематической проверки сформулированных гипотез определим следующие исследовательские вопросы:

RQ1. *Каково влияние дообучения на метрики классификации?* — Количественная оценка изменений accuracy, macro F1, precision и recall по классам t_{GEO} и t_{TAG} между базовой моделью f_{θ_0} и дообученными моделями f_{θ^*} на бенчмарках `benchmark-combined` (500 примеров: 326 GEO / 174 TAG) и `benchmark-real` (250 примеров: 203 GEO / 47 TAG).

RQ2. *Как зависит качество классификации от объёма обучающих данных?* — Анализ зависимости метрик от размера обучающей выборки при фиксированном составе и качестве данных.

RQ3. *Как тип обучающих данных влияет на робастность модели?* — Сравнение моделей, обученных на синтетических данных, реальных пользовательских запросах и их комбинациях, с точки зрения обобщающей способности и устойчивости к изменению распределения входных данных.

RQ4. *Статистически значимы ли различия между моделями?* — Применение критериев Макнемара и bootstrap-доверительных интервалов для попарного сравнения моделей.

2.4. Критерии успеха

Критерии успеха определяются на основании baseline-метрик базовой модели YandexGPT 5.1 Pro и целевых значений, обоснованных требованиями прикладной системы. Результаты сведены в Таблице 1.

Таблица 1: Критерии успеха: baseline-метрики базовой модели и целевые значения

Метрика	**Baseline**	**Цель**	**Бенчмарк**
Accuracy	81,8%	$> 90\%$	combined
Macro F1	0,526	$> 0{,}85$	combined
Recall (t_{TAG})	49,4%	$> 75\%$	combined
Без tool_call ($\rho_{\varnothing}$)	6,8%	$< 3\%$	combined

Низкий Recall для класса t_{TAG} и дисбаланс Macro F_1 (Табл. 1) отражают систематическое предпочтение базовой моделью класса GEO и являются основной мотивацией для дообучения.

2.5. Ограничения исследования

Сформулируем ключевые ограничения, в рамках которых проводится данное исследование:

1. **Фиксированная архитектура модели.** Исследование ограничено семейством моделей YandexGPT 5.1 (Lite и Pro). Результаты не обязаны переноситься на модели других архитектур и провайдеров.

2. **Бинарная постановка.** Рассматривается только два инструмента из множества инструментов реальной системы. Вопрос масштабирования на $|\mathcal{T}| > 2$ выходит за рамки работы.

3. **Дисбаланс классов.** Оба бенчмарка содержат существенное преобладание класса GEO (65% в `combined`, 81% в `real`), что отражает реальное распределение пользовательских запросов, но ограничивает статистическую мощность выводов по классу TAG.

4. **Управляемость LoRA.** Внутренние гиперпараметры дообучения (ранг r, α, learning rate, расписание, число шагов) управляются платформой Yandex Cloud и недоступны для прямого задания. Это ограничивает возможность выводов о влиянии конкретных

гиперпараметров и воспроизводимость результатов на других инфраструктурах.

5. **Привязка к версии базовой модели.** Результаты получены на YandexGPT 5.1 в период февраль–март 2026 г. Стабильность результатов при обновлении базовой модели провайдером не гарантируется.

6. **Межэкспертное согласие разметки.** Первичная разметка выполнена автором с консенсусным разрешением спорных случаев; формальный коэффициент Cohen's κ не вычислялся ввиду ограниченной доступности независимых разметчиков.

3. Обзор литературы

В настоящем разделе систематизируются основные результаты, относящиеся к предметной области исследования: методы параметрически эффективного дообучения больших языковых моделей, гипотеза поверхностного выравнивания, а также подходы к построению AI-агентов, способных использовать внешние инструменты. Обзор формирует теоретическую базу для последующей постановки эксперимента и интерпретации результатов.

3.1. Методы дообучения больших языковых моделей

В качестве базовой платформы в настоящей работе используется семейство моделей YandexGPT [32] — генеративные авторегрессионные модели на основе архитектуры Transformer [2, 17], разработанные компанией Яндекс и доступные в вариантах различного масштаба (YandexGPT Lite, YandexGPT Pro). Эти модели доступны через API Yandex Cloud и поддерживают дообучение на пользовательских данных.

Полное дообучение (full fine-tuning) предобученной языковой модели предполагает обновление всех её параметров на целевом наборе данных. Для моделей, содержащих миллиарды параметров, такой подход сопряжён с высокими вычислительными затратами, необходимостью хранения отдельной копии параметров для каждой задачи и риском катастрофического забывания [21]. В ответ на эти ограничения был разработан класс методов, объединённых под общим названием **Parameter-Efficient Fine-Tuning** (PEFT). Обзор данного направления представлен в работе Ding и др. [22], опубликованной в Nature Machine Intelligence. Авторы систематизируют существующие подходы к параметрически эффективному дообучению и выделяют несколько основных стратегий: добавление небольших обучаемых модулей к фиксированной модели, модификация лишь малого чис-

ла существующих параметров, а также обучение дополнительных представлений на входе модели.

Метод **LoRA** (Low-Rank Adaptation), предложенный Hu и др. [19], параметризует обновление весов через произведение двух матриц низкого ранга, что позволяет обучать менее 1% параметров модели при качестве, сопоставимом с полным дообучением. Формальное описание метода приведено в разделе 4.3.

Метод **Prompt Tuning** [18] добавляет обучаемые непрерывные векторы ко входу модели, оставляя её параметры замороженными. Подход **Adapter Layers** [23] встраивает небольшие обучаемые модули между слоями замороженной модели.

3.2. Гипотеза поверхностного выравнивания

Важным вопросом при дообучении языковых моделей является характер изменений, которые оно вносит: приобретает ли модель принципиально новые знания или лишь корректирует форму выдачи уже усвоенной при предобучении информации?

В работе Zhou и др. [16] была сформулирована **гипотеза поверхностного выравнивания** (Superficial Alignment Hypothesis). Авторы продемонстрировали, что модель LIMA, дообученная всего на 1000 тщательно отобранных примерах, способна генерировать ответы высокого качества, сопоставимые с результатами моделей, обученных на значительно больших наборах данных с использованием обучения с подкреплением на основе обратной связи от человека (RLHF) [31]. Согласно данной гипотезе, практически все фактические знания модель приобретает на этапе предобучения, тогда как дообучение с инструкциями (instruction fine-tuning) преимущественно обучает модель *формату и стилю* взаимодействия с пользователем. Иными словами, выравнивание является «поверхностным»: оно затрагивает не глубинные знания модели, а лишь способ их представления.

Эти выводы согласуются с результатами работы Gudibande и др. [12], в которой было показано, что модели, дообученные на выходах

более мощных проприетарных моделей (так называемая имитация), улучшают стиль генерации, но не приобретают фактических знаний, присущих моделям-учителям. Модели-имитаторы демонстрируют улучшение качества ответов по субъективным оценкам (стиль, связность), однако на объективных бенчмарках, измеряющих фактическую корректность, прирост отсутствует [12].

Для настоящего исследования гипотеза поверхностного выравнивания имеет ключевое значение. Если дообучение преимущественно корректирует стиль, а не знания, то выбор инструмента в контексте AI-агента может быть интерпретирован как элемент «стиля» генерации — устойчивый паттерн оформления ответа, который модель способна усвоить из относительно небольшого числа качественных примеров. Данная интерпретация обосновывает ожидание того, что целенаправленное дообучение на компактном, но качественном наборе данных способно значимо улучшить точность классификации инструментов.

3.3. AI-агенты и выбор инструментов

Понятие AI-агента на основе большой языковой модели подразумевает систему, в которой LLM выступает в роли центрального «мыслящего» компонента, способного планировать действия, обращаться к внешним инструментам и интерпретировать результаты их выполнения. Обзорная работа Xi и др. [25] систематизирует архитектуры таких агентов и выделяет три ключевых компонента: модуль восприятия (perception), модуль планирования и рассуждений (brain), а также модуль действия (action), включающий взаимодействие с внешними инструментами.

Одним из наиболее влиятельных подходов к организации рассуждений агента является фреймворк **ReAct**, предложенный Yao и др. [24]. ReAct объединяет генерацию цепочки рассуждений (reasoning) и выполнение действий (acting) в чередующемся цикле «мысль — действие — наблюдение». Модель последовательно формулирует рассуждение о текущей ситуации, выполняет действие (например, вызов API или поисковый запрос), получает наблюдение (результат действия) и использует

его для следующего шага рассуждений. Такая архитектура позволяет агенту адаптивно выбирать инструменты на каждом шаге, исходя из контекста задачи.

В работе Schick и др. [30] был предложен подход **Toolformer**, в котором языковая модель самостоятельно обучается вставлять вызовы внешних инструментов (калькулятор, поисковая система, переводчик и др.) в генерируемый текст. Обучение осуществляется в формате self-supervised: модель генерирует кандидатные вызовы инструментов, фильтрует их по критерию снижения перплексии и дообучается на отфильтрованных данных. Toolformer продемонстрировал, что модели способны научиться определять, *когда* и *какой* инструмент следует использовать, без явной разметки.

Работа Patil и др. [14] непосредственно связана с задачей настоящего исследования. Авторы предложили модель **Gorilla**, дообученную для корректного вызова API из обширного каталога. Gorilla обучалась на наборе данных, сгенерированном с помощью Self-Instruct [26], и продемонстрировала способность точно генерировать вызовы API, превосходя по этому показателю модели GPT-4 без специализированного дообучения. Данный результат подтверждает гипотезу о том, что целенаправленное дообучение на задачу выбора и вызова инструментов способно существенно повысить качество классификации.

Обзорная работа Qin и др. [28] систематизирует направление обучения моделей использованию инструментов (tool learning). Авторы выделяют четыре ключевых этапа взаимодействия модели с инструментами: понимание задачи, планирование использования инструментов, непосредственный вызов инструмента и обработка его результата. Особый интерес представляет выделенный авторами этап *выбора инструмента* (tool selection) — задача классификации, в которой модель должна из доступного набора инструментов выбрать наиболее подходящий для решения текущей подзадачи. Именно эта задача является предметом настоящего исследования.

Открытые бенчмарки tool use. Для сравнительной оценки моделей в задачах function calling разработан ряд англоязычных бенч-

марков. *Berkeley Function Calling Leaderboard* (BFCL) [3] предоставляет стандартизированные наборы задач на одиночный и множественный вызов API и параллельное исполнение; *ToolBench* [29] включает более 16 тыс. реальных API и используется для оценки обобщающей способности моделей; *API-Bank* [1] сосредоточен на этапах планирования и многошагового взаимодействия. Эти бенчмарки покрывают широкий спектр англоязычных API, но не содержат задач русскоязычного геодомена, в котором функционирует исследуемый AI-агент. В настоящей работе используется собственный русскоязычный бенчмарк, отражающий специфику предметной области (разделы 5.2–5.2.2).

Выбор YandexGPT. YandexGPT выбрана как модель, применяемая в промышленной эксплуатации исследуемого AI-агента; результаты дообучения данной модели напрямую переносимы в production. Сравнение с альтернативными русскоязычными LLM (GigaChat, Saiga, ruGPT) выходит за рамки настоящей работы и обозначено в направлениях дальнейших исследований (раздел 9).

3.4. Синтетическая генерация обучающих данных

Подготовка качественных обучающих данных для дообучения языковых моделей является ресурсоёмкой задачей. Метод **Self-Instruct**, предложенный Wang и др. [26], предлагает подход к автоматической генерации обучающих примеров с использованием самой языковой модели. Процедура включает итеративную генерацию инструкций, входных данных и эталонных ответов с последующей фильтрацией некачественных и дублирующихся примеров. Авторы продемонстрировали, что дообучение модели на синтетически сгенерированных данных приводит к значимому улучшению качества следования инструкциям, приближаясь к результатам моделей, обученных на данных, размеченных вручную.

Результаты Self-Instruct [26] согласуются с выводами LIMA [16]: если выравнивание является поверхностным и затрагивает преимущественно формат генерации, то для его достижения достаточно относительно

небольшого числа качественных примеров, даже если эти примеры получены синтетическим путём. Для задачи классификации инструментов это означает, что эффективный обучающий набор может быть сформирован с помощью LLM-генерации при условии обеспечения разнообразия сценариев и корректности разметки. Модель Gorilla [14] является практическим подтверждением данного подхода: её обучающий набор был получен методом Self-Instruct.

3.5. Выводы обзора

Проведённый обзор литературы позволяет сформулировать следующие ключевые наблюдения. Во-первых, методы параметрически эффективного дообучения, в особенности LoRA [19], позволяют адаптировать модели при минимальных вычислительных затратах [22]. Во-вторых, гипотеза поверхностного выравнивания [16, 12] обосновывает ожидание того, что дообучение способно значимо улучшить паттерны генерации при небольшом объёме данных. В-третьих, работы Toolformer [30] и Gorilla [14] демонстрируют эффективность дообучения для задачи выбора инструментов [25, 28].

Вместе с тем, **открытым остаётся вопрос** о том, каким образом дообучение модели масштаба YandexGPT Lite влияет на качество классификации инструментов в реальной агентной системе, каков минимальный объём обучающих данных, достаточный для значимого улучшения, и в какой мере наблюдаемый эффект согласуется с гипотезой поверхностного выравнивания. Настоящая работа направлена на экспериментальное исследование именно этих вопросов.

4. Математические основы

В настоящем разделе вводятся формальные определения и математический аппарат, необходимые для строгой постановки задачи классификации инструментов, оценки качества моделей и статистического сравнения результатов.

4.1. Формализация задачи бинарной классификации

Пусть $\mathcal{X}$ — пространство признаков, элементами которого являются текстовые запросы пользователей к AI-агенту. Определим множество меток $\mathcal{Y} = \{0, 1\}$, где метка 0 соответствует классу `TAG` (тегирование), а метка 1 — классу `GEO` (геокодирование).

Определение 4 (Классификатор)**.** *Классификатором называется измеримое отображение $f_\theta \colon \mathcal{X} \to \mathcal{Y}$, параметризованное вектором параметров $\theta \in \Theta$, где Θ — пространство параметров модели.*

Пусть дана выборка $\{(x_i, y_i)\}_{i=1}^n$, где $x_i \in \mathcal{X}$, $y_i \in \mathcal{Y}$. Обозначим предсказания классификатора $\hat{y}_i = f_\theta(x_i)$.

Определение 5 (Матрица ошибок)**.** *Матрицей ошибок (confusion matrix) классификатора f_θ на выборке $\{(x_i, y_i)\}_{i=1}^n$ называется набор из четырёх величин:*

$$\mathrm{TP} = \big|\{i : \hat{y}_i = 1 \ \wedge \ y_i = 1\}\big|, \tag{1}$$

$$\mathrm{FP} = \big|\{i : \hat{y}_i = 1 \ \wedge \ y_i = 0\}\big|, \tag{2}$$

$$\mathrm{TN} = \big|\{i : \hat{y}_i = 0 \ \wedge \ y_i = 0\}\big|, \tag{3}$$

$$\mathrm{FN} = \big|\{i : \hat{y}_i = 0 \ \wedge \ y_i = 1\}\big|, \tag{4}$$

где $|\cdot|$ обозначает мощность множества. Очевидно, $\mathrm{TP} + \mathrm{FP} + \mathrm{TN} + \mathrm{FN} = n$.

Положительным классом в дальнейшем считается `GEO` ($y = 1$); предметная интерпретация двух типов ошибок ($\mathrm{TAG} \to \mathrm{GEO}$ и $\mathrm{GEO} \to \mathrm{TAG}$) для рассматриваемого AI-агента приведена в разделе 2.

4.2. Метрики качества классификации

Пусть $\mathcal{C} = \{0, 1\}$ — множество классов, $|\mathcal{C}| = 2$. Для каждого класса $c \in \mathcal{C}$ обозначим через TP_c, FP_c, TN_c, FN_c элементы матрицы ошибок, построенной в стратегии «один против всех» (one-vs-rest) для класса c.

Определение 6 (Accuracy)**.** *Доля правильных ответов (accuracy) классификатора определяется как*

$$\mathrm{Acc} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}. \tag{5}$$

Определение 7 (Precision)**.** *Точность (precision) для класса $c \in \mathcal{C}$ определяется как*

$$P_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}. \tag{6}$$

Величина P_c характеризует долю истинных представителей класса c среди всех объектов, отнесённых классификатором к классу c.

Определение 8 (Recall)**.** *Полнота (recall) для класса $c \in \mathcal{C}$ определяется как*

$$R_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}. \tag{7}$$

Величина R_c характеризует долю верно распознанных представителей класса c среди всех объектов, истинно принадлежащих классу c.

Определение 9 (F_1-мера)**.** *F_1-мера для класса c определяется как среднее гармоническое точности и полноты:*

$$F_{1,c} = \frac{2\,P_c\,R_c}{P_c + R_c} = \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}. \tag{8}$$

Для обобщения метрик на многоклассовый случай (а также для единообразного представления результатов бинарной классификации) используются агрегированные варианты F_1-меры.

Определение 10 (Macro F_1)**.** *Макро-усреднённая F_1-мера определяется как среднее арифметическое F_1-мер по всем классам:*

$$F_1^{\mathrm{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} F_{1,c}. \tag{9}$$

Данная метрика придаёт равный вес каждому классу вне зависимости от его представленности в выборке.

Определение 11 (Weighted F_1)**.** *Взвешенная F_1-мера определяется как*

$$F_1^{\text{weighted}} = \sum_{c \in \mathcal{C}} \frac{n_c}{N} F_{1,c}, \tag{10}$$

где $n_c = |\{i : y_i = c\}|$ — число объектов класса c в выборке, $N = \sum_{c \in \mathcal{C}} n_c$ — общий объём выборки. Эта метрика учитывает дисбаланс классов.

Систематический анализ метрик классификации и их свойств приведён в [27].

4.3. Низкоранговая адаптация (LoRA)

Метод низкоранговой адаптации (Low-Rank Adaptation, LoRA) [19] позволяет эффективно дообучать предобученные языковые модели, модифицируя лишь малую часть параметров.

Пусть $W_0 \in \mathbb{R}^{d \times k}$ — матрица весов некоторого линейного слоя предобученной модели. В процессе полного дообучения (full fine-tuning) вычисляется обновление $\Delta W \in \mathbb{R}^{d \times k}$, после чего используется матрица $W = W_0 + \Delta W$. Количество обучаемых параметров при этом составляет $d \cdot k$.

Ключевая идея LoRA состоит в параметризации обновления через произведение двух матриц малого ранга:

$$W = W_0 + \Delta W = W_0 + B\,A, \tag{11}$$

где $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, а $r \ll \min(d, k)$ — ранг адаптации.

Определение 12 (Низкоранговая адаптация)**.** *Низкоранговой адаптацией с рангом r называется замена обновления $\Delta W \in \mathbb{R}^{d \times k}$ на произведение $\Delta W = B\,A$, где $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, при фиксированной исходной матрице W_0.*

Количество обучаемых параметров при использовании LoRA для

одного слоя составляет

$$r\,(d+k) \tag{12}$$

вместо $d \cdot k$. При $r \ll \min(d,k)$ достигается значительное сокращение числа параметров:

$$\frac{r\,(d+k)}{d \cdot k} = r\left(\frac{1}{k} + \frac{1}{d}\right) \ll 1. \tag{13}$$

Инициализация матриц выполняется следующим образом:

$$A_{ij} \sim \mathcal{N}(0, \sigma^2), \qquad B = \mathbf{0}_{d \times r}, \tag{14}$$

что гарантирует $\Delta W = BA = \mathbf{0}$ на момент начала обучения. Таким образом, дообучение начинается с исходной предобученной модели без внесения начальных возмущений.

Прямой проход через адаптированный слой для входного вектора $h \in \mathbb{R}^k$ вычисляется как

$$h' = W_0\, h + \frac{\alpha}{r}\, B\, A\, h, \tag{15}$$

где $\alpha > 0$ — масштабирующий гиперпараметр, контролирующий вклад адаптации. Множитель α/r обеспечивает стабильность обучения при изменении ранга r.

4.4. Статистические критерии сравнения классификаторов

Для статистически обоснованного сравнения нескольких классификаторов, обученных на одних и тех же данных и оценённых на общей тестовой выборке, необходимо применять специализированные критерии, учитывающие зависимость предсказаний.

4.4.1. Критерий Q Кохрена

Критерий Q Кохрена [7] применяется для проверки гипотезы об однородности $k \geqslant 2$ бинарных классификаторов, оценённых на одной и

той же выборке из n объектов.

Пусть $X_{ij} \in \{0, 1\}$ — индикатор правильного ответа j-го классификатора ($j = 1, \ldots, k$) на i-м объекте ($i = 1, \ldots, n$). Введём обозначения:

$$T_j = \sum_{i=1}^{n} X_{ij}, \qquad R_i = \sum_{j=1}^{k} X_{ij}, \qquad \bar{T} = \frac{1}{k} \sum_{j=1}^{k} T_j. \tag{16}$$

Здесь T_j — количество правильных ответов j-го классификатора, R_i — количество классификаторов, давших правильный ответ на i-й объект.

Определение 13 (Нулевая гипотеза критерия Кохрена)**.** *H_0: все k классификаторов имеют одинаковую вероятность правильного ответа, т. е. $P(X_{i1} = 1) = P(X_{i2} = 1) = \ldots = P(X_{ik} = 1)$ для всех i.*

Статистика критерия Кохрена определяется формулой

$$Q = \frac{k(k-1) \sum_{j=1}^{k} (T_j - \bar{T})^2}{k \sum_{i=1}^{n} R_i - \sum_{i=1}^{n} R_i^2}. \tag{17}$$

При справедливости H_0 и достаточно большом n статистика Q имеет асимптотическое распределение хи-квадрат:

$$Q \xrightarrow{d} \chi^2(k-1). \tag{18}$$

Гипотеза H_0 отвергается при $Q > \chi^2_{1-\alpha}(k-1)$, где $\chi^2_{1-\alpha}(k-1)$ — $(1-\alpha)$-квантиль распределения χ^2 с $k-1$ степенями свободы. Отвержение H_0 означает, что хотя бы один классификатор статистически значимо отличается от остальных по доле правильных ответов.

4.4.2. Критерий Макнемара

Для попарного сравнения двух классификаторов A и B, оценённых на одной и той же выборке из n объектов, применяется критерий Макнемара [20, 8].

Построим таблицу сопряжённости 2×2 по результатам работы двух классификаторов:

	B верно	B неверно
A верно	n_{11}	n_{10}
A неверно	n_{01}	n_{00}

(19)

где n_{10} — количество объектов, на которых A ответил верно, а B — неверно; n_{01} — количество объектов, на которых B ответил верно, а A — неверно. Величины n_{11} и n_{00} (согласованные ответы) не несут информации о различии классификаторов.

Определение 14 (Нулевая гипотеза критерия Макнемара)**.** *H_0: вероятности дискордантных исходов равны, т. е.*

$$P(A \text{ верно, } B \text{ неверно}) = P(B \text{ верно, } A \text{ неверно}). \tag{20}$$

При малых значениях $n_{10} + n_{01}$ (что типично для задач с ограниченными тестовыми выборками) используется точный биномиальный тест. При справедливости H_0 случайная величина n_{10} имеет биномиальное распределение:

$$n_{10} \sim \mathrm{Bin}\big(n_{10} + n_{01},\ \tfrac{1}{2}\big)\,. \tag{21}$$

p-значение вычисляется как вероятность наблюдения столь же или более экстремального результата:

$$p = 2\,\min\Big(P\big(X \leqslant n_{10}\big),\ P\big(X \geqslant n_{10}\big)\Big), \quad X \sim \mathrm{Bin}\big(n_{10} + n_{01},\ \tfrac{1}{2}\big)\,. \tag{22}$$

При больших $n_{10} + n_{01}$ может быть использовано асимптотическое приближение со статистикой

$$\chi^2_{\mathrm{McN}} = \frac{(n_{10} - n_{01})^2}{n_{10} + n_{01}}, \tag{23}$$

имеющей при H_0 распределение $\chi^2(1)$.

4.4.3. Поправка Бонферрони

При проведении m попарных сравнений возникает проблема множественных проверок: вероятность хотя бы одного ложноположительного результата (family-wise error rate, FWER) возрастает с ростом m.

Определение 15 (FWER)**.** *Групповая вероятность ошибки первого рода определяется как*

$$\mathrm{FWER} = P\big(\textit{хотя бы одно ложное отвержение } H_0\big). \tag{24}$$

Поправка Бонферрони [5, 9] является простейшим способом контроля FWER. При проведении m одновременных проверок скорректированный уровень значимости каждого отдельного теста полагается равным

$$\alpha_{\mathrm{adj}} = \frac{\alpha}{m}, \tag{25}$$

где α — желаемый уровень FWER. Неравенство Бонферрони гарантирует, что

$$\mathrm{FWER} = P\left(\bigcup_{i=1}^{m} A_i\right) \leqslant \sum_{i=1}^{m} P(A_i) = m \cdot \alpha_{\mathrm{adj}} = \alpha, \tag{26}$$

где A_i — событие ложного отвержения H_0 в i-м тесте.

Правило применения. Каждое из m попарных сравнений проводится критерием Макнемара, после чего его p-значение сравнивается не с исходным α, а с $\alpha_{\mathrm{adj}} = \alpha/m$; эквивалентно, нулевая гипотеза для отдельного теста отвергается, если $p \leqslant \alpha_{\mathrm{adj}}$, либо, что то же самое, если скорректированное значение $p_{\mathrm{adj}} = \min(1,\ m \cdot p)$ не превосходит α.

В нашем случае при сравнении k классификаторов попарно число сравнений составляет $m = \binom{k}{2} = \frac{k(k-1)}{2}$. Для $k = 12$ получаем $m = 66$ и $\alpha_{\mathrm{adj}} = 0{,}05/66 \approx 0{,}000758$.

Поправка Бонферрони консервативна, особенно при положительной корреляции между тестами (что характерно для попарных сравнений на одной и той же выборке): фактический FWER, как правило, оказывается строго меньше α, а статистическая мощность снижается. Менее

консервативные альтернативы — например, последовательная процедура Холма [15] — также контролируют FWER на уровне α, но обладают большей мощностью. В настоящей работе мы сознательно выбираем процедуру Бонферрони, поскольку основной вывод формулируется в форме «модель A значимо превосходит модель B», и для такого утверждения предпочтительнее минимизировать риск ложного отвержения, нежели терять часть истинных различий.

4.5. Бутстрап доверительные интервалы

Для оценки неопределённости метрик качества классификации применяется метод непараметрического бутстрапа [10], не требующий предположений о параметрическом виде распределения метрики.

Пусть $\mathcal{D} = \{(x_i, y_i, \hat{y}_i)\}_{i=1}^n$ — тестовая выборка с истинными и предсказанными метками. Пусть $\theta = T(\mathcal{D})$ — значение интересующей метрики (например, F_1, accuracy), вычисленное на выборке $\mathcal{D}$.

Определение 16 (Бутстрап-выборка)**.** *Бутстрап-выборкой $\mathcal{D}_b^*$ ($b = 1, \ldots, B$) называется выборка объёма n, полученная из $\mathcal{D}$ случайным выбором с возвращением:*

$$\mathcal{D}_b^* = \big\{(x_{i_1}, y_{i_1}, \hat{y}_{i_1}),\ \ldots,\ (x_{i_n}, y_{i_n}, \hat{y}_{i_n})\big\}, \quad i_1, \ldots, i_n \overset{\text{iid}}{\sim} \text{Uniform}\{1, \ldots, n\}. \tag{27}$$

Алгоритм построения бутстрап доверительного интервала перцентильным методом состоит из следующих шагов.

1. Для каждого $b = 1, \ldots, B$ сформировать бутстрап-выборку $\mathcal{D}_b^*$ и вычислить $\theta_b^* = T(\mathcal{D}_b^*)$.

2. Упорядочить бутстрап-оценки: $\theta_{(1)}^* \leqslant \theta_{(2)}^* \leqslant \ldots \leqslant \theta_{(B)}^*$.

3. Построить доверительный интервал уровня $(1 - \alpha)$ как

$$\text{CI}_{1-\alpha} = \Big[\theta_{(\lfloor B\alpha/2 \rfloor)}^*,\ \theta_{(\lceil B(1-\alpha/2) \rceil)}^*\Big]. \tag{28}$$

В настоящей работе используется $B = 1000$ бутстрап-выборок и уровень доверия $1-\alpha = 0{,}95$, что обеспечивает построение 95% доверительных интервалов для всех исследуемых метрик.

Обоснование состоятельности перцентильного метода основано на принципе подстановки (plug-in principle): эмпирическое распределение $\hat{F}_n$ аппроксимирует истинное распределение F, а распределение $T(\mathcal{D}^*) - T(\mathcal{D})$ аппроксимирует распределение $T(\mathcal{D}) - \theta$, где θ — истинное значение метрики. Состоятельность аппроксимации $\hat{F}_n \to F$ при $n \to \infty$ обеспечивает классическая теорема Гливенко–Кантелли [13, 6]:

$$\sup_{x \in \mathbb{R}} \left| \hat{F}_n(x) - F(x) \right| \xrightarrow{\text{п. н.}} 0,$$

а состоятельность бутстрап-оценки распределения статистики T при выполнении условий регулярности (липшицевость, конечность дисперсии) показана в [4, 11]. Тестовые выборки настоящей работы ($n = 500$ и $n = 250$) удовлетворяют этим условиям для рассматриваемых метрик.

5. Методология экспериментов

В данном разделе описывается методология экспериментальной части исследования: пайплайн дообучения и оценки моделей, состав бенчмарков и обучающих датасетов, протокол дообучения, а также статистический дизайн эксперимента.

5.1. Пайплайн дообучения и оценки

Экспериментальный процесс организован в виде шестиэтапного пайплайна, обеспечивающего воспроизводимость и систематичность проведения экспериментов. Общая схема пайплайна представлена на Рис. 1.

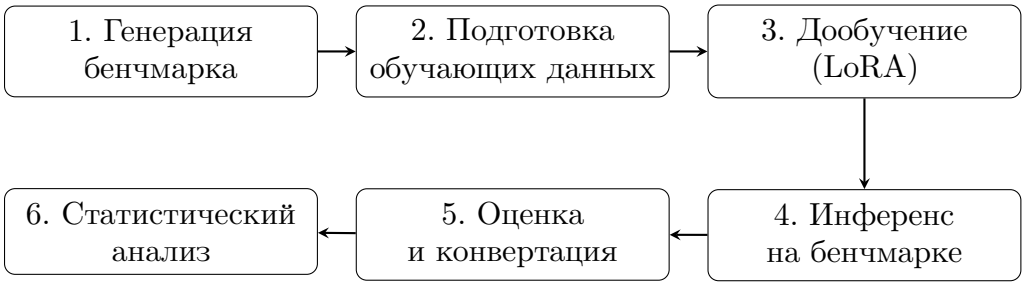
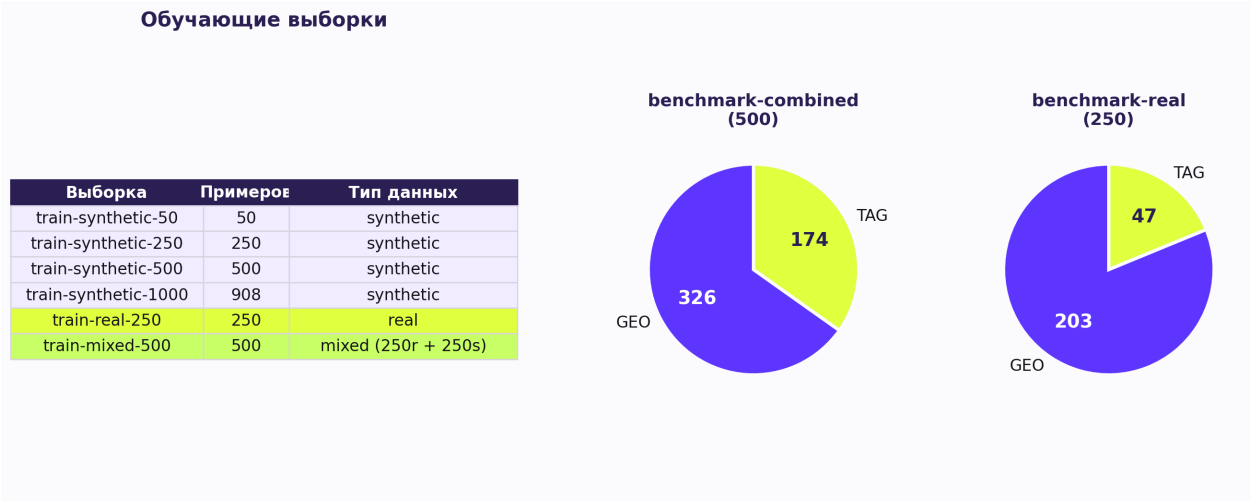


Рис. 1: Пайплайн дообучения и оценки моделей.



Выборка	Примеров	Тип данных
train-synthetic-50	50	synthetic
train-synthetic-250	250	synthetic
train-synthetic-500	500	synthetic
train-synthetic-1000	908	synthetic
train-real-250	250	real
train-mixed-500	500	mixed (250r + 250s)

Рис. 2: Дизайн эксперимента: 12 моделей, 2 бенчмарка, 24 цикла оценки; статистический анализ через критерии Кохрена Q, Макнемара (с поправкой Бонферрони) и bootstrap-CI.

Каждый этап пайплайна выполняет следующие функции:

1. **Генерация бенчмарка.** Формирование оценочных наборов данных двух типов: смешанного (реальные и синтетические запросы) и реального (только пользовательские запросы). Каждый пример содержит пользовательский запрос и эталонную метку инструмента (GEO или TAG).

2. **Подготовка обучающих данных.** Создание обучающих датасетов различного объёма, типа и классового баланса. Синтетические данные генерируются по методологии Self-Instruct [26] с последующей гуманизацией для приближения к стилистике реальных запросов.

3. **Дообучение (LoRA).** Адаптация базовой модели YandexGPT Lite методом LoRA [19] через API Yandex Cloud [32]. На выходе — дообученная модель с уникальным идентификатором чекпоинта.

4. **Инференс на бенчмарке.** Последовательный прогон каждого примера из бенчмарка через модель с фиксированными параметрами генерации. Результат — XML-разметка с вызовом инструмента.

5. **Оценка и конвертация.** Парсинг ответов модели, извлечение имени вызванного инструмента из тегов `<tool_call>`, сопоставление с эталонной меткой. Вычисление метрик accuracy, precision, recall, F1 для каждого класса.

6. **Статистический анализ.** Применение непараметрических критериев (Cochran Q, McNemar) и бутстрап-оценок доверительных интервалов для проверки статистической значимости различий между моделями.

5.2. Описание бенчмарков

Для оценки качества классификации инструментов используются два бенчмарка, различающихся по составу и назначению (Табл. 2).

Таблица 2: Характеристики оценочных бенчмарков.

Бенчмарк	Объём	GEO	TAG	Тип данных
`combined`	500	326 (65,2%)	174 (34,8%)	Реал. + синтет.
`real`	250	203 (81,2%)	47 (18,8%)	Только реальные

benchmark-combined (500 примеров) содержит смесь реальных пользовательских запросов и синтетических примеров. Соотношение классов GEO/TAG составляет 326/174, что отражает естественное преобладание географических запросов в предметной области. Данный бенчмарк предназначен для оценки общей способности модели к классификации инструментов.

benchmark-real (250 примеров) состоит исключительно из реальных пользовательских запросов, собранных из логов AI-агента. Распределение классов существенно смещено: 203 запроса GEO (81,2%) против 47 запросов TAG (18,8%). Данный бенчмарк предназначен для оценки качества модели в условиях, максимально приближённых к промышленной эксплуатации.

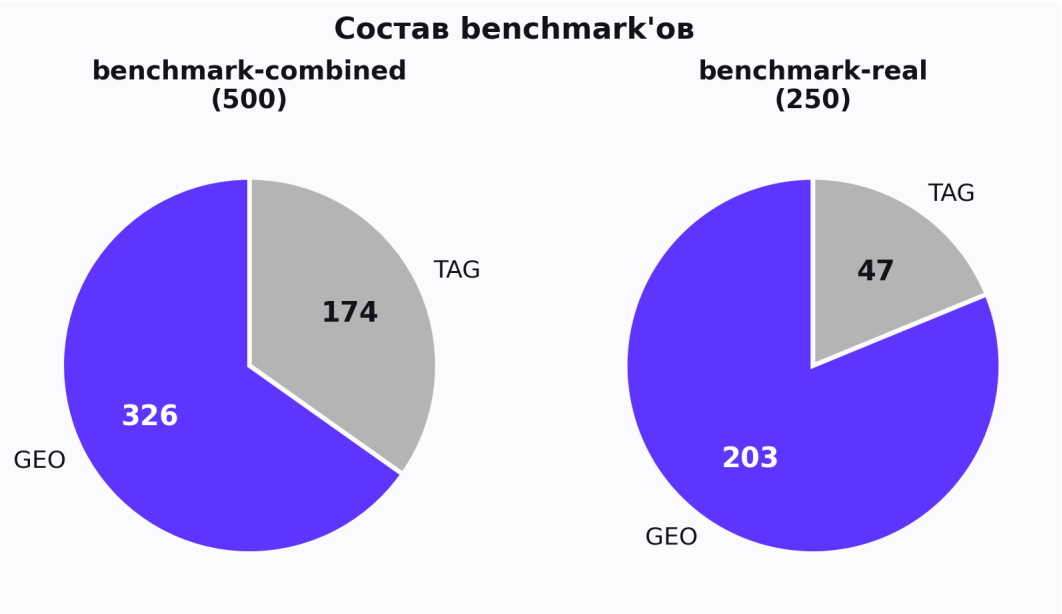


Рис. 3: Состав и классовый баланс оценочных бенчмарков.

5.2.1. Политика разметки пограничных случаев

При создании бенчмарков применялась единая политика разметки, определяющая отнесение запроса к классу GEO или TAG в неоднозначных ситуациях:

- **GEO**: запрос содержит конкретные имена собственные — названия станций метро («ст. м. Сокол»), улиц («ул. Тверская»), районов, городов, конкретных адресов. Критерий — наличие именованной географической сущности, требующей разрешения через геокодер.

- **TAG**: запрос описывает класс инфраструктуры или характеристику объекта без привязки к конкретному имени собственному. Примеры: «рядом с парком», «с камином», «у метро» (как категория транспортной доступности, без указания конкретной станции).

Ключевое разграничение: выражение «у метро Сокол» классифицируется как GEO (содержит имя собственное), тогда как «у метро» — как TAG (описывает характеристику без конкретной привязки).

5.2.2. Сбор реальных данных и контроль утечек

Источник. Реальные пользовательские запросы получены из логов промышленного AI-агента в домене поиска недвижимости. Из лог-пула был сформирован унифицированный набор из 500 уникальных (по нормализованному тексту) запросов, удовлетворяющих политике разметки.

Обезличивание и этика. Все запросы прошли автоматическое обезличивание: удалены идентификаторы пользователя, номера телефонов, адреса электронной почты и иные личные данные. Пользователи давали согласие на использование запросов в исследовательских целях. Ни один запрос не содержит личной информации в публикуемой версии датасета.

Разметка. Первичная разметка выполнена автором работы; спорные пограничные случаи (нарушающие единообразное применение по-

литики из раздела выше) прошли второй раунд разметки с привлечением двух независимых помощников; итоговая метка устанавливалась консенсусом после обсуждения. Формальное межэкспертное согласие (Cohen's κ) не вычислялось; это отмечено как ограничение (раздел 2).

Контроль утечек. Из 500 уникальных реальных запросов 250 включены в `benchmark-real` (тестовая выборка), оставшиеся 250 — в обучающий датасет `train-real-250`. Перед дообучением выполнена проверка пересечения обеих выборок по нормализованному тексту запроса: `benchmark-real` $\cap$ `train-real-250` $= \varnothing$. Аналогичная проверка выполнена между `benchmark-combined` и всеми обучающими датасетами; утечек не обнаружено. Таким образом, все результаты оценки получены на строго независимых от обучения примерах.

5.3. Обучающие датасеты

Для исследования влияния объёма и типа обучающих данных на качество классификации были подготовлены шесть основных обучающих датасетов (Табл. 3).

Таблица 3: Характеристики обучающих датасетов.

Датасет	Объём	Тип	GEO	TAG	Гуман.
`synth-50`	50	Синтет.	54,0%	46,0%	Да
`synth-250`	250	Синтет.	53,6%	46,4%	Да
`synth-500`	500	Синтет.	55,8%	44,2%	Да
`synth-1000`	908	Синтет.	52,0%	48,0%	Да
`real-250`	250	Реал.	82,4%	17,6%	Нет
`mixed-500`	500	Реал. + синт.	68,0%	32,0%	Нет

Синтетические датасеты (`train-synthetic-*`) сгенерированы по методологии Self-Instruct [26] и прошли процедуру гуманизации — автоматическое преобразование, приближающее стилистику и лексику запросов к реальным пользовательским формулировкам. Классовый баланс в синтетических датасетах поддерживается на уровне, близком к 50/50.

Датасет `train-real-250` содержит только реальные пользовательские запросы с естественным распределением классов (82,4% GEO / 17,6% TAG), что отражает доминирование географических запросов в предметной области.

Датасет `train-mixed-500` представляет собой объединение реальных и синтетических примеров, что позволяет исследовать эффект смешивания типов данных при обучении.

Выбор такого набора датасетов мотивирован необходимостью ответить на следующие исследовательские вопросы: (а) как объём обучающей выборки влияет на качество (серия `synthetic-50/250/500/1000`); (б) превосходят ли реальные данные синтетические при равном объёме (`real-250` vs `synthetic-250`); (в) даёт ли смешивание типов данных преимущество (`mixed-500` vs `synthetic-500`).

Обучающие выборки

Выборка	Примеров	Тип данных
train-synthetic-50	50	synthetic
train-synthetic-250	250	synthetic
train-synthetic-500	500	synthetic
train-synthetic-1000	908	synthetic
train-real-250	250	real
train-mixed-500	500	mixed (250r + 250s)

Рис. 4: Объёмы и классовый состав обучающих датасетов.

Помимо основных шести датасетов, в эксперимент включены пять дополнительных моделей (`lite_latest_v{1,2,3}` и `lite_rc_v{1,2}`), представляющих собой промежуточные итерации разработки. Каждая из них обучена на собственном корпусе объёмом порядка 1500 синтетических примеров; корпуса различаются между собой вследствие стохастичности LLM-генерации и последующей дедупликации дубликатов

по нормализованному тексту запроса, поэтому объёмы после дедупликации варьируются (итоговые корпуса не сохранялись отдельно от соответствующих моделей). Эти модели включены в анализ не как независимые конфигурации с контролируемой вариацией фактора, а как эмпирическая оценка устойчивости подхода при варьировании обучающего корпуса того же порядка величины.

5.4. Протокол дообучения

5.4.1. Метод дообучения

Дообучение выполняется методом LoRA [19] (раздел 4.3) через API Yandex Cloud [32]. Yandex Cloud API предоставляет дообучение как управляемый сервис: пользователь загружает обучающий датасет в формате JSONL, указывает базовую модель и запускает задачу обучения. Внутренние гиперпараметры LoRA (ранг r, коэффициент масштабирования α, learning rate) управляются платформой и недоступны для прямого задания пользователем. При очень малом объёме обучающей выборки дообучение может не полностью покрыть пространство допустимых имён инструментов, что в отдельных случаях приводит к генерации вызовов несуществующих функций; это явление наблюдается у модели, обученной на 50 примерах (раздел 6.4).

5.4.2. Базовая модель

В качестве базовой модели для дообучения используется YandexGPT Lite — компактная версия семейства YandexGPT, оптимизированная для задач с низкой латентностью. Для сравнения в роли baseline выступает более крупная модель YandexGPT 5.1 Pro (`pro_baseline`), используемая без дообучения.

5.4.3. Конфигурация инференса

При проведении инференса на бенчмарках для всех моделей применяются единые параметры генерации:

- **Temperature**: 0,1 — низкое значение для снижения стохастичности сэмплирования; полная детерминированность генерации на стороне платформы Yandex Cloud не гарантируется, поэтому каждый эксперимент выполнен в *одном* прогоне без усреднения;
- **Max tokens**: 4096 — верхняя граница длины генерируемого ответа;
- **Retry policy**: при временных сбоях API (rate limit, 5xx) запрос повторяется с экспоненциальной задержкой; постоянные сбои (после исчерпания попыток) фиксируются как $\rho_{\varnothing}$ — ответы без `tool_call`;
- **Парсинг**: извлечение имени вызванного инструмента из XML-тегов `<tool_call>`.

5.5. Дизайн эксперимента

5.5.1. Перечень моделей

В эксперименте участвуют 12 моделей: 1 baseline-модель без дообучения и 11 дообученных вариантов YandexGPT Lite. Полный перечень приведён в Табл. 4.

Таблица 4: Перечень моделей, участвующих в эксперименте.

Модель	**База**	**Обучающий датасет**	**Объём**
`pro_baseline`	GPT 5.1 Pro	— (без дообучения)	—
`train_synth_50`	GPT Lite	`synth-50`	50
`train_synth_250`	GPT Lite	`synth-250`	250
`train_synth_500`	GPT Lite	`synth-500`	500
`train_synth_1000`	GPT Lite	`synth-1000`	908
`train_real_250`	GPT Lite	`real-250`	250
`train_mixed_500`	GPT Lite	`mixed-500`	500
`lite_latest_v1`	GPT Lite	Синт. (итер. 1)*	∼1500
`lite_latest_v2`	GPT Lite	Синт. (итер. 2)*	∼1500
`lite_latest_v3`	GPT Lite	Синт. (итер. 3)*	∼1500
`lite_rc_v1`	GPT Lite	Синт. (итер. 4)*	∼1500
`lite_rc_v2`	GPT Lite	Синт. (итер. 5)*	∼1500

* промежуточные итерации разработки; каждая обучена на собственном корпусе ~ 1500 синтетических примеров, различающихся после дедупликации, см. раздел выше.

5.5.2. Матрица экспериментов

Каждая из 12 конфигураций моделей оценивается на каждом из 2 бенчмарков, что даёт $12 \times 2 = 24$ цикла оценки (evaluation runs). Для каждого цикла фиксируются бинарные исходы (верно/неверно) по каждому примеру бенчмарка, что обеспечивает возможность применения попарных статистических тестов на связанных выборках.

5.5.3. Статистические тесты

Для анализа результатов применяется трёхуровневая схема статистического тестирования (формальные определения всех критериев приведены в разделе 4.4).

Глобальный тест. Критерий Кохрена Q [7] проверяет нулевую гипотезу о равенстве вероятностей правильной классификации для всех k моделей. При отклонении H_0 выполняются попарные сравнения.

Попарные сравнения. При отклонении глобальной гипотезы Кохрена для каждой из $\binom{12}{2} = 66$ пар моделей строится таблица сопряжённости и применяется точный критерий Макнемара [20]. Поскольку все 66 тестов проводятся одновременно и на одной и той же тестовой выборке, при работе на номинальном уровне $\alpha = 0{,}05$ вероятность хотя бы одного ложного отвержения (FWER) могла бы достигать $1 - (1 - 0{,}05)^{66} \approx 0{,}97$. Для контроля FWER применяется поправка Бонферрони [5, 9]: p-значение каждого теста сравнивается не с α, а со скорректированным порогом $\alpha_{\text{adj}} = 0{,}05/66 \approx 0{,}000758$. Различие между моделями признаётся значимым, только если $p \leqslant \alpha_{\text{adj}}$; формальное обоснование — в разделе 4.4.3.

Доверительные интервалы. Для оценки accuracy каждой модели строятся 95%-доверительные интервалы методом бутстрапа [10] ($B = 1000$, seed $= 42$).

5.5.4. Контроль воспроизводимости

Для обеспечения воспроизводимости результатов приняты следующие меры:

- Фиксированное значение temperature $= 0{,}1$ при инференсе для минимизации стохастичности генерации.
- Фиксированное зерно seed $= 42$ для всех процедур бутстрапа.
- Сохранение уникальных идентификаторов чекпоинтов всех дообученных моделей в Yandex Cloud.
- Сохранение полных логов инференса (запрос, ответ модели, извлечённая метка) для каждого эксперимента.
- Использование единых бенчмарков без перемешивания или ресемплирования между запусками.

6. Результаты экспериментов и их анализ

В данном разделе приводятся результаты экспериментальной оценки 12 моделей на двух независимых бенчмарках: `benchmark-combined` (500 примеров: 326 GEO / 174 TAG) и `benchmark-real` (250 примеров: 203 GEO / 47 TAG). Методология подготовки данных и проведения экспериментов описана в разделе 5; определения метрик — в разделе 4.

6.1. Оценка базовой модели

Базовой линией (baseline) служит модель YandexGPT 5.1 Pro без дообучения (`pro_baseline`). Результаты представлены в таблице 5.

Таблица 5: Метрики базовой модели `pro_baseline` на двух бенчмарках. 95% bootstrap-CI для accuracy построен перцентильным методом ($B = 1000$, seed = 42).

Бенчмарк	**Accuracy**	**95% CI**	**Macro F1**	**Ошибки**	$\boldsymbol{\rho_\varnothing}$
combined (500)	0,818	[0,784; 0,852]	0,526	57	6,8%
real (250)	0,828	[0,784; 0,872]	0,617	28	6,0%

Низкое значение Macro $F_1 = 0{,}526$ при относительно высокой accuracy 0,818 указывает на выраженный дисбаланс качества между классами. Анализ матрицы ошибок выявляет систематический паттерн: все 57 ошибок на `benchmark-combined` представляют собой ошибки типа TAG→GEO, то есть запросы класса TAG массово классифицируются как GEO. Ни одного обратного случая (GEO→TAG) не зафиксировано.

Данный bias объясняется тем, что базовая модель не дообучена для данного набора инструментов и по умолчанию предпочитает вызов `ResolveEntities` (GEO) как более общий. Помимо ошибок классификации, для 34 запросов (6,8%) модель не генерирует `tool_call` вовсе ($\rho_\varnothing = 6{,}8\%$), что является критическим сбоем в контексте AI-агента.

6.2. Основные результаты дообучения

Результаты всех 12 моделей на `benchmark-combined` приведены в таблице 6. Модели различаются по типу и объёму обучающих данных.

Таблица 6: Результаты на `benchmark-combined` (500 примеров: 326 GEO / 174 TAG). 95% bootstrap-CI для accuracy построен перцентильным методом ($B = 1000$, seed $= 42$); $\rho_\varnothing$ — доля ответов без `tool_call`.

Модель	**Обуч. данные**	**Acc**	**95% CI**	**M-F1**	**Ош.**	$\rho_\varnothing$
pro_baseline	— (без тюна)	.818	[.784; .852]	.526	57	6,8%
lite_latest_v1	Синт. ∼1500	.928	[.904; .948]	.925	34	0,4%
lite_latest_v2	Синт. ∼1500	.984	[.972; .994]	.985	6	0,4%
lite_latest_v3	Синт. ∼1500	.966	[.950; .982]	.966	15	0,4%
lite_rc_v1	Синт. ∼1500	.980	[.966; .992]	.981	8	0,4%
lite_rc_v2	Синт. ∼1500	.974	[.958; .988]	.974	11	0,4%
train_synth_50	Синт. 50	.946	[.926; .964]	.631	25	0,4%
train_synth_250	Синт. 250	.946	[.926; .964]	.945	25	0,4%
train_synth_500	Синт. 500	.978	[.964; .990]	.979	9	0,4%
train_synth_1000	Синт. 908	.956	[.938; .974]	.955	20	0,4%
train_real_250	Реал. 250	.952	[.932; .970]	.949	22	0,4%
train_mixed_500	Микс 500	.994	[.986; 1,000]	.996	1	0,4%

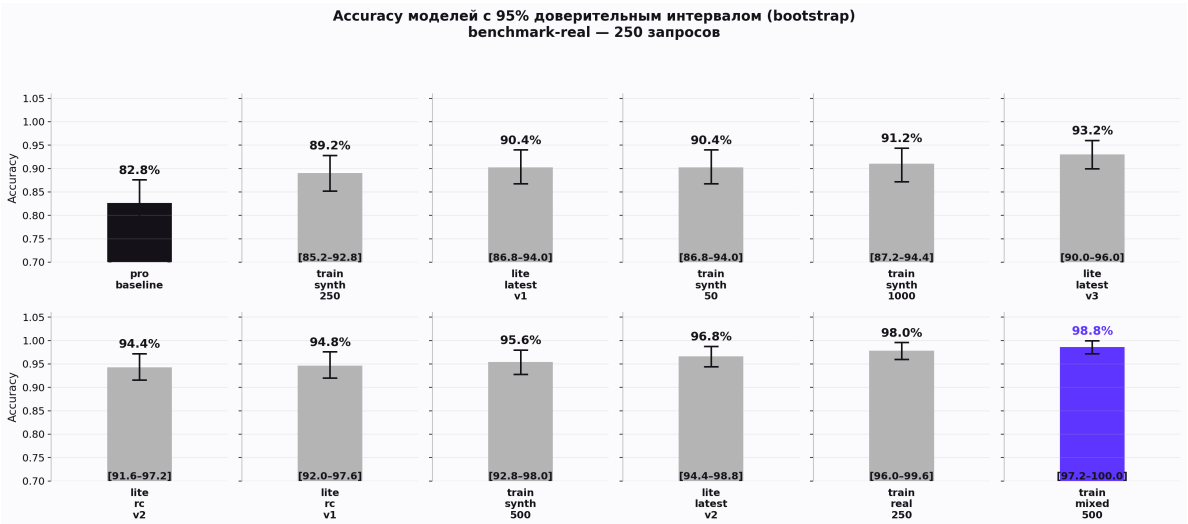


Рис. 5: Accuracy всех 12 моделей на `benchmark-combined` с 95% bootstrap-доверительными интервалами. Горизонтальная линия — целевое значение 0,9.

Ключевые наблюдения:

1. **Скачок при минимальном объёме данных.** Даже 50 синтетических примеров повышают accuracy с 0,818 до 0,946 (+12,8 п.п.), подтверждая гипотезу 1: даже минимальный объём примеров радикально улучшает качество, что согласуется с гипотезой поверхностного выравнивания [16].

2. **Устранение отказов.** Все дообученные модели снижают $\rho_{\varnothing}$ с 6,8% (34 из 500) до 0,4% (2 из 500).

3. **Абсолютный лидер.** Модель `train_mixed_500`, обученная на сбалансированной комбинации реальных и синтетических данных (500 примеров), достигает accuracy 0,994 и Macro $F_1 = 0{,}996$ с единственной ошибкой классификации.

6.3. Валидация на реальных данных

Для оценки обобщающей способности моделей используется независимый бенчмарк `benchmark-real`, составленный из реальных пользовательских запросов. Результаты и разность с `benchmark-combined` приведены в таблице 7.

Таблица 7: Результаты на `benchmark-real` (250 примеров: 203 GEO / 47 TAG). 95% bootstrap-CI — перцентильный метод, $B = 1000$, seed = 42; $\rho_{\varnothing}$ — доля ответов без `tool_call`; ΔAcc — разность с `benchmark-combined`.

Модель	**Acc**	**95% CI**	**M-F1**	**Ош.**	$\boldsymbol{\rho_{\varnothing}}$	**ΔAcc**
pro_baseline	0,828	[.784; .872]	0,617	28	6,0%	+0,010
lite_latest_v1	0,904	[.868; .936]	0,872	22	0,8%	−0,024
lite_latest_v2	0,968	[.944; .988]	0,959	6	0,8%	−0,016
lite_latest_v3	0,932	[.900; .960]	0,906	15	0,8%	−0,034
lite_rc_v1	0,948	[.916; .972]	0,921	11	0,8%	−0,032
lite_rc_v2	0,944	[.912; .968]	0,916	12	0,8%	−0,030
train_synth_50	0,904	[.868; .936]	0,584	22	0,8%	−0,042
train_synth_250	0,892	[.852; .928]	0,856	25	0,8%	−0,054
train_synth_500	0,956	[.928; .980]	0,942	9	0,8%	−0,022
train_synth_1000	0,912	[.876; .944]	0,882	20	0,8%	−0,044
train_real_250	0,980	[.960; .996]	0,978	3	0,8%	+0,028
train_mixed_500	0,988	[.972; 1,000]	0,991	1	0,8%	−0,006

Столбец ΔAcc показывает разность accuracy на `benchmark-real` и `benchmark-combined`. Модели, обученные исключительно на синтетических данных, демонстрируют устойчивое снижение accuracy на реальных данных в диапазоне $-2{,}2 \ldots -5{,}4$ п.п. Это свидетельствует об ограниченной обобщающей способности синтетических данных: модели частично переобучаются на характерные паттерны генерации, отсутствующие в реальных запросах.

Модель `train_real_250`, обученная на 250 реальных примерах, напротив, демонстрирует прирост +2,8 п.п. при переходе на реальный бенчмарк, что объяснимо совпадением распределений обучающей и тестовой выборок.

Модель `train_mixed_500` показывает минимальную деградацию: $\Delta\text{Acc} = -0{,}006$, что свидетельствует о высокой робастности, достигаемой за счёт сочетания разнородных обучающих данных.

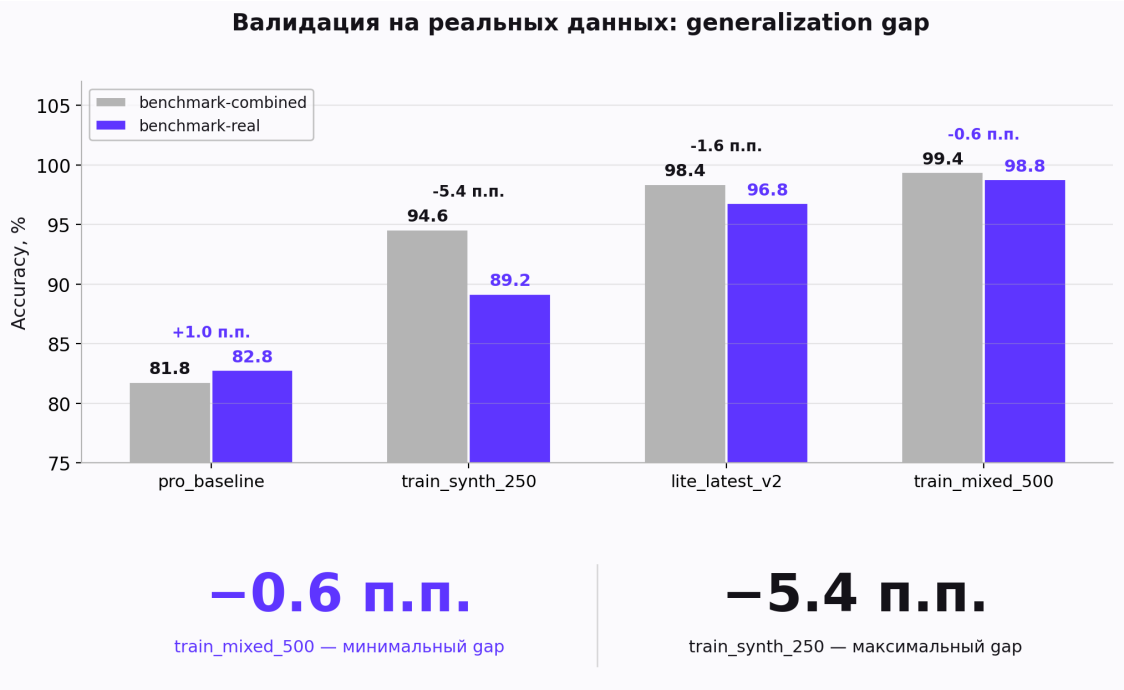


Рис. 6: Обобщающий разрыв: Δ accuracy при переходе с `benchmark-combined` на `benchmark-real`. Отрицательные значения — деградация на реальных данных.

6.4. Анализ кривой обучения

Для исследования зависимости качества от объёма обучающих данных (RQ2) обучена серия моделей на синтетических выборках объёмом 50, 250, 500 и 908 примеров. Результаты на `benchmark-combined` представлены в таблице 8.

Таблица 8: Кривая обучения: зависимость метрик от объёма синтетических данных на `benchmark-combined`. 95% bootstrap-CI ($B = 1000$, seed = 42, перцентильный метод).

Объём	Accuracy	95% CI	Macro F1
50	0,946	[0,926; 0,964]	0,631
250	0,946	[0,926; 0,964]	0,945
500	0,978	[0,964; 0,990]	0,979
908	0,956	[0,938; 0,974]	0,955

Кривая обучения обнаруживает нетривиальный немонотонный ха-

рактер:

1. **Разрыв accuracy и Macro F_1 при 50 примерах.** При одинаковой accuracy (0,946) на 50 и 250 примерах Macro F_1 возрастает с 0,631 до 0,945. Этот разрыв объясняется не деградацией распознавания, а артефактом macro-усреднения. Анализ confusion matrix модели `train_synth_50` (таблица 9) показывает, что в 2 случаях из 500 модель генерирует вызов *несуществующего* инструмента `GetSearchQuerySummary` — побочный продукт недоученной модели, не покрывшей всё пространство допустимых токенов. В отчёте оценщика этот лейбл фиксируется как третий класс с support $= 0$ и $F_1 = 0$, что тянет среднее вниз:

   $$\text{Macro}\, F_1 = (0 + 0{,}959 + 0{,}934)/3 = 0{,}631.$$

   На 250 примерах паразитный класс исчезает, оценщик возвращается к двум лейблам, и Macro F_1 нормализуется до 0,945. Таким образом, уже 50 примеров достаточно для устранения основного bias baseline, однако минимум ~ 250 примеров необходимо для устойчивого подавления генерации несуществующих имён инструментов.

2. **Пик на 500 примерах.** Максимальное качество среди синтетических моделей достигается при 500 примерах: accuracy $= 0{,}978$, Macro $F_1 = 0{,}979$.

3. **Деградация при увеличении до 908 примеров.** Дальнейшее наращивание объёма приводит к снижению accuracy до 0,956 ($-2{,}2$ п.п.). Такой эффект объясняется переобучением на артефакты синтетической генерации: с ростом объёма выборки доля повторяющихся шаблонов увеличивается, и модель начинает эксплуатировать статистические закономерности, специфичные для синтетического распределения.

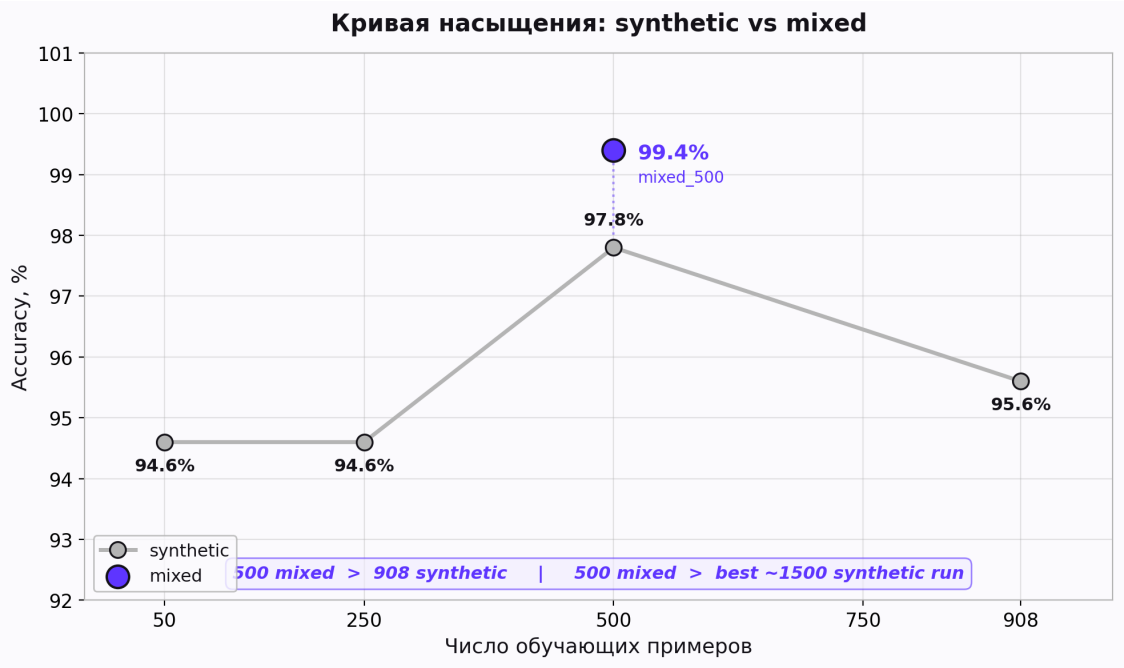


Рис. 7: Кривая обучения: зависимость accuracy и Macro F_1 от объёма синтетических обучающих данных на `benchmark-combined`.

Таблица 9: Confusion matrix модели `train_synth_50` на `benchmark-combined`. Строки — истинный класс, столбцы — предсказание модели; $n_\varnothing$ — количество ответов без `tool_call`.

Истинно \ Предсказано	`ResolveEntities`	`SearchTags`	`GetSearchQ`
`ResolveEntities` (GEO, 326)	304	19	
`SearchTags` (TAG, 174)	4	169	

* несуществующий инструмент, ошибочно сгенерированный моделью.

6.5. Анализ типа обучающих данных

Для проверки гипотезы 3 сравним модели, обученные на данных различного типа при сопоставимых объёмах. Результаты сведены в таблице 10.

Таблица 10: Сравнение моделей по типу обучающих данных

Модель	**Тип данных**	**combined**		**real**	
		Acc	M-F1	Acc	M-F1
train_synth_500	Синтет. 500	0,978	0,979	0,956	0,942
train_real_250	Реал. 250	0,952	0,949	0,980	0,978
train_mixed_500	Микс 500	0,994	0,996	0,988	0,991

Наблюдается отчётливая специализация моделей по типу обучающих данных:

- Модель на синтетике (`train_synth_500`) превосходит модель на реальных данных (`train_real_250`) на смешанном бенчмарке (+2,6 п.п.), но уступает ей на реальном (−2,4 п.п.).
- `train_real_250` показывает лучший результат на `benchmark-real` (0,980), но её accuracy на `benchmark-combined` ниже (0,952), поскольку синтетические примеры в тестовой выборке имеют иную структуру.
- `train_mixed_500` объединяет преимущества обоих типов данных и доминирует на обоих бенчмарках: 0,994 и 0,988 соответственно.

Данный результат подтверждает гипотезу 3: качество (разнородность и репрезентативность) обучающих данных оказывает большее влияние на итоговые метрики, чем их объём. Модель `train_mixed_500`, обученная на 500 примерах, превосходит все модели, обученные на ~ 1500 синтетических примерах.

6.6. Анализ ошибок

6.6.1. Паттерны ошибок классификации

Для каждой модели определены направления ошибок: GEO→TAG (запрос GEO ошибочно классифицирован как TAG) и TAG→GEO (запрос TAG ошибочно классифицирован как GEO). Результаты представлены в таблице 11.

Таблица 11: Паттерны ошибок классификации на `benchmark-combined`

Модель	**G→T**	**T→G**	**Домин. ошибка**
pro_baseline*	0	57	TAG массово как GEO
train_synth_*	8–24	1	GEO массово как TAG
train_real_250	1	21	TAG→GEO (дисбал.)
train_mixed_500	0	1	Практически нет

* значение «0 ошибок GEO→TAG» не отражает преимущества baseline: модель склонна по умолчанию предпочитать GEO, поэтому обратных ошибок не возникает. Кроме того, 34 запроса (6,8%) не содержат `tool_call` вовсе и не учтены в столбцах G→T / T→G.

Обнаруживается характерная инверсия bias: базовая модель систематически классифицирует TAG-запросы как GEO, тогда как дообученные на синтетике модели совершают преимущественно обратную ошибку (GEO→TAG). Это объясняется тем, что синтетические обучающие данные перепредставляют класс TAG, смещая решающую границу модели.

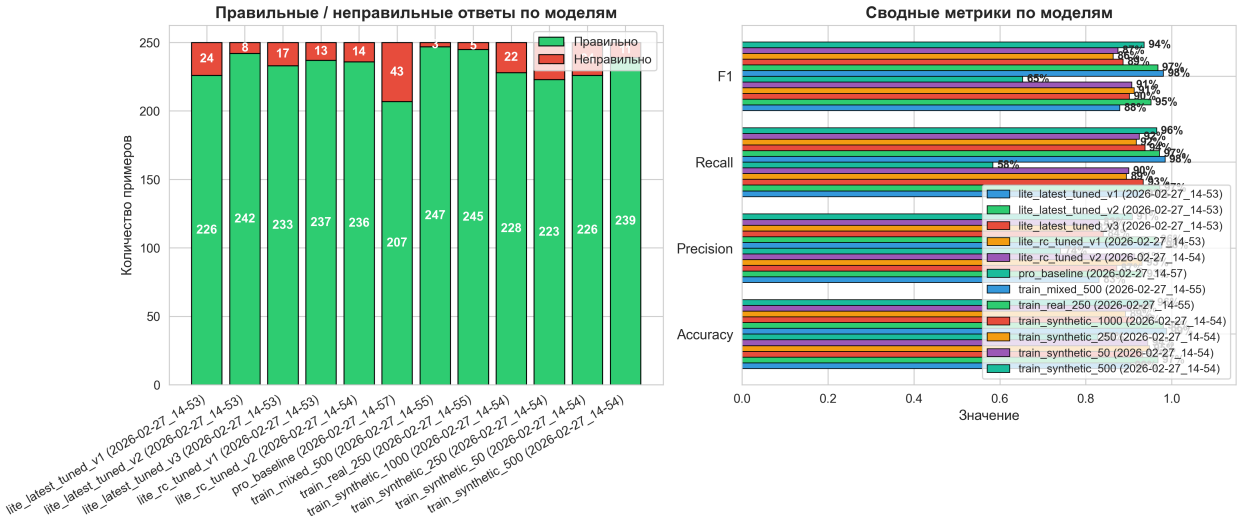


Рис. 8: Количество ошибок по моделям с разбиением по направлению (GEO→TAG, TAG→GEO, отказ) на обоих бенчмарках.

Модель `train_mixed_500` практически свободна от обоих типов bias (0 ошибок GEO→TAG, 1 ошибка TAG→GEO), что подтверждает эффективность сбалансированного микса данных для устранения систематических смещений.

6.6.2. Консервация ошибок

Анализ пересечения ошибок выявляет важный феномен: многие ошибки воспроизводимы — и на уровне одной модели (между бенчмарками), и между разными моделями.

Воспроизводимость между бенчмарками. Для трёх показательных моделей (таблица 12) ошибки на `benchmark-real` в подавляющем большинстве совпадают с ошибками на `benchmark-combined`. Это означает, что ошибки обусловлены не стохастикой генерации, а систематическими слабостями модели на определённых типах запросов.

Таблица 12: Консервация ошибок: совпадение ошибок на двух бенчмарках (combined / real)

Модель	**GEO→TAG**	**TAG→GEO**
lite_latest_v2	5 / 5	1 / 1
train_synth_500	8 / 8	1 / 1
train_mixed_500	0 / 0	1 / 1

Распределение ошибок по 12 моделям. На `benchmark-combined` из 500 примеров 368 (73,6%) классифицированы правильно *всеми* 12 моделями; ещё 78 (15,6%) ошибочны лишь у одной модели. Ни один пример не ошибочен у всех 12 моделей — это означает, что каждый запрос принципиально решаем хотя бы одной из рассмотренных конфигураций. В то же время существует устойчивое «трудное ядро»: 9 примеров ошибочны у шести и более моделей, 3 примера — у одиннадцати из двенадцати (таблица 13).

Таблица 13: Распределение примеров `benchmark-combined` по числу ошибающихся моделей

Ошиблось моделей	**Кол-во примеров**	**% от 500**
0	368	73,6
1	78	15,6
2–3	35	7,0
4–5	10	2,0
6–8	6	1,2
9–11	6	1,2
12	0	0,0

Пересечение ошибочных множеств между моделями. Коэффициент Жаккара между множествами ошибочных ответов двух моделей, $J(A, B) = |E_A \cap E_B|/|E_A \cup E_B|$, даёт меру близости их паттернов ошибок. Наиболее характерные наблюдения:

- $J(\texttt{pro_baseline}, \cdot) \leqslant 0{,}17$ для всех дообученных моделей — ошибки baseline *качественно отличны* от ошибок дообученных моделей (у baseline это TAG→GEO, у дообученных — GEO→TAG, см. таблицу 11).

- $J(\texttt{lite_rc_v1}, \texttt{lite_rc_v2}) = 0{,}77$ — почти идентичные множества ошибок, что естественно для моделей одной итерации разработки.

- $J(\texttt{train_synth_50}, \texttt{train_synth_1000}) = 0{,}58$ — существенное совпадение ошибок у синтетических моделей разного объёма, подтверждающее, что артефакты синтетической генерации консервативны.

- $J(\texttt{train_real_250}, \texttt{train_mixed_500}) = 0{,}12$ — различные паттерны ошибок у моделей с реальными данными.

Лингвистический анализ «трудного ядра». Из 9 примеров, ошибочных у шести и более моделей, 8 относятся к классу GEO. Все они

обладают одним или несколькими общими признаками (таблица 14):

Таблица 14: Примеры «трудного ядра»: запросы, ошибочные у $\geqslant 9$ моделей из 12

ID	True	Запрос	Ошиб.
q_00159	GEO	«купить однокомнатную в центре Симферополя»	11
q_00165	TAG	«кладовка от застройщика новый век»	11
q_00253	GEO	«купить квартиру ялта ул красноармейская 13»	11
q_00046	GEO	«ищу квартиру поближе к зелёной ветке и подешевле снять»	9
q_00473	GEO	«...с очень классным ремонтом..., в рамках ТТК, до 80 000 руб.»	9
q_00496	GEO	«снять студию с собакой в приморском районе...»	9

Выделяются два систематических источника ошибок:

1. **Жаргонные и абревиатурные топонимы.** «Зелёная ветка» (линия метро), «ТТК» (Третье транспортное кольцо), «приморский район» (имя собственное в нижнем регистре) — модели не распознают эти выражения как именованные сущности и склоняют их к классу TAG.

2. **Длинные комплексные запросы с доминирующей TAG-составляющей.** В запросах типа q_00473 и q_00496 географическая привязка занимает менее 10% длины запроса, а остальные признаки (ремонт, этажность, цена, наличие животного) воспринимаются как TAG-характеристики; модели следуют за доминантой.

Обратный случай — q_00165 («кладовка от застройщика новый век») — представляет собой ошибку противоположного типа: «Новый век» является потенциальным названием жилого комплекса, и модели ошибочно распознают его как имя собственное, относя запрос к GEO.

Выявленные паттерны соответствуют фундаментально неоднозначным случаям: безусловно корректная классификация требует внешней

базы знаний (словарь имён собственных, справочник жилых комплексов), что выходит за пределы возможностей классификатора на основе LLM без RAG-компонента.

6.7. Статистическая значимость

6.7.1. Критерий Кохрена

Для проверки глобальной гипотезы о равенстве долей ошибок всех 12 моделей применён Q-критерий Кохрена (Cochran's Q test), являющийся обобщением критерия Макнемара на случай $k > 2$ связанных выборок (раздел 4):

- `benchmark-combined`: $Q = 348{,}0$, $p < 10^{-6}$;
- `benchmark-real`: $Q = 128{,}8$, $p < 10^{-6}$.

На обоих бенчмарках нулевая гипотеза $H_0\colon \pi_1 = \pi_2 = \ldots = \pi_{12}$ отвергается при любом разумном уровне значимости. Следовательно, модели статистически значимо различаются по доле ошибок.

6.7.2. Попарные сравнения (критерий Макнемара)

Для попарного сравнения моделей применён точный критерий Макнемара. При $k = 12$ моделях число пар составляет $m = \binom{12}{2} = 66$, поэтому каждое получаемое p-значение сравнивается со скорректированным порогом $\alpha_{\text{adj}} = 0{,}05/66 \approx 0{,}000758$ (поправка Бонферрони, раздел 4.4.3). Различие признаётся статистически значимым только при $p \leqslant \alpha_{\text{adj}}$; такая граница обеспечивает FWER не выше 0,05 для всей серии 66 сравнений.

`benchmark-combined`.

- `train_mixed_500` статистически значимо превосходит большинство моделей ($p < \alpha_{\text{adj}}$ для всех пар, кроме `lite_latest_v2`).
- Пара `lite_latest_v2` — `lite_rc_v1` неразличима: $p = 0{,}73 \gg \alpha_{\text{adj}}$.

`benchmark-real`.

- `train_mixed_500` и `train_real_250` статистически неразличимы: $p = 0{,}50$.
- Обе модели статистически значимо превосходят все модели, обученные исключительно на синтетических данных.

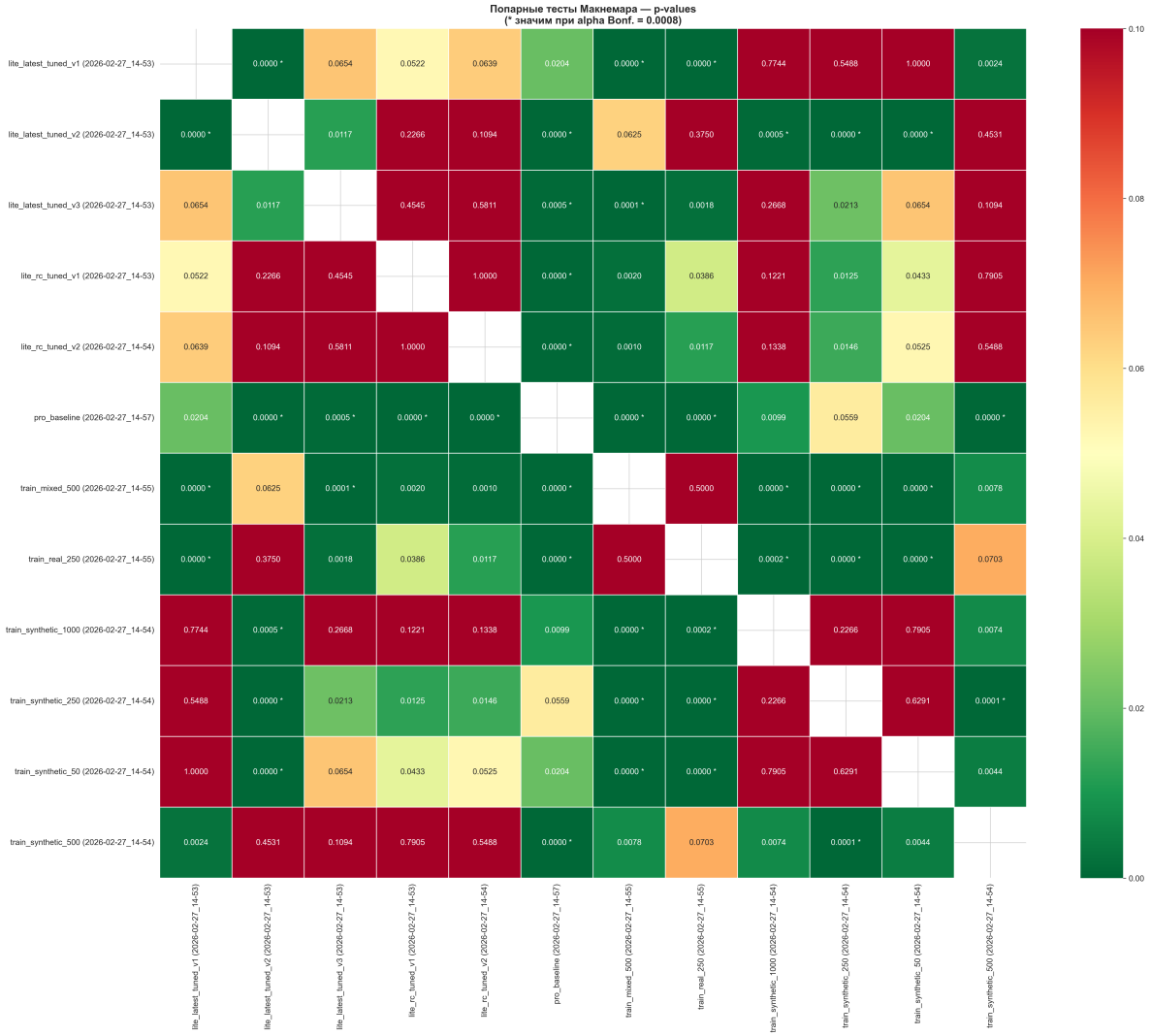


Рис. 9: Тепловая карта p-значений попарного критерия Макнемара для 12 моделей на `benchmark-combined`. Тёмные ячейки — значимые различия ($p < \alpha_{\text{adj}}$), светлые — неразличимые пары.

Данные результаты означают, что на реальных данных наличие реальных примеров в обучающей выборке является ключевым

фактором: модель `train_real_250` (250 реальных примеров) статистически неотличима от `train_mixed_500` (500 смешанных), тогда как `train_synth_500` (500 синтетических) значимо уступает обеим.

6.7.3. Согласованность предсказаний

Анализ согласованности позволяет оценить долю примеров, на которых все 12 моделей дают одинаковый ответ:

- `benchmark-combined`: 368 из 500 примеров (73,6%) классифицированы правильно всеми 12 моделями; 0 примеров ошибочны у всех 12.
- `benchmark-real`: 175 из 250 примеров (70,0%) классифицированы правильно всеми 12 моделями.

Отсутствие примеров, ошибочных у всех 12 моделей, означает, что каждый запрос в бенчмарке в принципе решаем хотя бы одной из рассмотренных моделей. Оставшиеся $\sim$27–30% примеров составляют «зону разногласий», где модели расходятся в предсказаниях, и именно на этих примерах сосредоточены различия в качестве.

6.8. Ответы на исследовательские вопросы

RQ1: Каково влияние дообучения на метрики классификации? Дообучение методом LoRA приводит к статистически значимому улучшению всех метрик. Лучшая модель `train_mixed_500` повышает accuracy с 0,818 до 0,994 (+17,6 п.п.) и Macro F_1 с 0,526 до 0,996 (+0,470). Даже наименее успешная дообученная модель (`lite_latest_v1`, accuracy = 0,928) превосходит baseline на +11,0 п.п. Число отказов ($\rho_\varnothing$) снижается с 6,8% до 0,4% для всех дообученных моделей. Результат статистически значим по критерию Кохрена ($Q = 348{,}0$, $p < 10^{-6}$) и попарным критериям Макнемара.

RQ2: Как зависит качество от объёма обучающих данных? Зависимость немонотонна. При фиксированном типе данных (синтетика)

accuracy растёт от 0,946 (50 примеров) до пика 0,978 (500 примеров), после чего снижается до 0,956 (908 примеров). Наиболее информативна динамика Macro F_1: скачок с 0,631 до 0,945 при переходе от 50 к 250 примерам указывает на то, что именно этот диапазон объёмов критичен для достижения сбалансированного распознавания обоих классов. Деградация при 908 примерах свидетельствует о переобучении на синтетическое распределение.

RQ3: Как тип обучающих данных влияет на робастность модели? Тип данных оказывает определяющее влияние. Модели на чистой синтетике демонстрируют деградацию $-2{,}2\ldots-5{,}4$ п.п. при переходе на реальные данные. Модель `train_real_250` показывает обратный эффект: $+2{,}8$ п.п. на `benchmark-real` по сравнению с `benchmark-combined`. Оптимальная стратегия — смешанный датасет: `train_mixed_500` достигает accuracy $\geq 0{,}988$ на обоих бенчмарках с деградацией всего $-0{,}6$ п.п. Это подтверждает гипотезу 3.

RQ4: Статистически значимы ли различия между моделями? Да. Критерий Кохрена отвергает гипотезу о равенстве моделей на обоих бенчмарках ($p < 10^{-6}$). Попарные сравнения Макнемара с поправкой Бонферрони ($\alpha_{\text{adj}} \approx 0{,}0008$) показывают, что `train_mixed_500` значимо превосходит большинство моделей на `benchmark-combined`. На `benchmark-real` модели `train_mixed_500` и `train_real_250` статистически неразличимы ($p = 0{,}50$) и обе значимо лучше всех чисто синтетических моделей. Пара `lite_latest_v2` — `lite_rc_v1` также неразличима ($p = 0{,}73$).

Проверка гипотез H1a и H1b. Гипотеза 1 (H1a) подтверждается попарным критерием Макнемара между `pro_baseline` и любой дообученной моделью: $p \ll \alpha_{\text{adj}}$. Гипотеза 2 (H1b) подтверждается непосредственно: нижняя граница 95%-го bootstrap-CI accuracy для `train_mixed_500` на `benchmark-combined` составляет 0,986 (таблица 6), что существенно превышает требуемый порог 0,90.

7. Угрозы валидности

В настоящем разделе обсуждаются факторы, способные повлиять на интерпретацию полученных результатов. Обсуждение структурировано по четырём классическим категориям: внутренняя, внешняя, конструктная и статистическая валидность.

7.1. Внутренняя валидность

Один прогон инференса. Каждая модель оценена на каждом бенчмарке в одном прогоне без усреднения. Сэмплирование при `temperature` $= 0{,}1$ не является строго детерминированным, что вносит остаточную стохастичность в число ошибок на уровне отдельных единиц. Эффект консервации ошибок (раздел 6.6), зафиксированный при воспроизведении запусков, указывает на то, что большая часть ошибок систематична; однако для моделей, различия между которыми близки к ошибке измерения (например, `lite_latest_v2` и `lite_rc_v1` с $p = 0{,}73$), вывод об эквивалентности следует принимать с учётом этой неопределённости.

Неуправляемые гиперпараметры LoRA. Ранг r, коэффициент α, learning rate и расписание обучения контролируются платформой Yandex Cloud и не доступны напрямую. Следовательно, делать выводы о влиянии конкретных гиперпараметров LoRA на результат нельзя; наблюдаемые эффекты отражают совокупный результат «данные $\times$ платформенная конфигурация».

Субъективность разметки. Первичная разметка выполнена одним автором; спорные случаи обсуждались с двумя помощниками. Формальное межэкспертное согласие (Cohen's κ) не вычислялось. Это означает, что часть ошибок моделей на пограничных запросах (см. «трудное ядро» в разделе 6.6) может отражать неоднозначность самой разметки.

7.2. Внешняя валидность

Одно семейство моделей. Все эксперименты проведены на YandexGPT 5.1 (Lite и Pro). Переносимость выводов на модели других архитектур и провайдеров (GigaChat, GPT-4o, Llama, Qwen) не гарантируется.

Узкий домен и язык. Результаты получены на русскоязычных запросах в домене поиска недвижимости. Обобщение на другие домены (медицина, юриспруденция, e-commerce) требует отдельной валидации, особенно в части «трудного ядра», которое в нашем случае состоит преимущественно из топонимов-аббревиатур и сложных составных запросов.

Бинарность постановки. В работе рассматривается выбор между двумя инструментами. При $|\mathcal{T}| > 2$ задача усложняется (растёт число возможных ошибок, снижается эффективная точность случайного угадывания), и количественные выводы — прежде всего минимальный объём обучающих примеров — не обязаны сохраняться.

7.3. Конструктная валидность

Что именно измеряется. Accuracy и Macro F_1 вычислены по результату классификации *первого* вызова инструмента (`tool_call`) в ответе модели. Качество последующего многошагового взаимодействия (передача корректных аргументов, обработка результата инструмента, диалоговая связность) не оценивается. Высокое значение accuracy гарантирует корректный выбор функции, но не гарантирует корректность её применения.

Метрика «правильности». Ошибка I и II рода определены в терминах классификации (раздел 4), а не в терминах удовлетворённости пользователя. В промышленной системе цена ошибки типа TAG→GEO и GEO→TAG может быть различна; данное различие в настоящей работе не учитывается.

7.4. Статистическая валидность

Ограниченный объём выборок. Бенчмарки содержат 500 и 250 примеров; для оценки модели с accuracy $\geqslant 0{,}99$ это означает всего несколько ошибок в абсолютном выражении. Попарные сравнения на таких уровнях точности близки к границе статистической мощности, что ограничивает способность обнаруживать малые, но реальные различия между топ-моделями.

Консервативность поправки Бонферрони. При $\binom{12}{2} = 66$ попарных сравнениях поправка Бонферрони обеспечивает FWER $\leqslant 0{,}05$, но предположение о независимости тестов нарушается (оценки моделей коррелируют, поскольку вычислены на общей выборке). Как следствие, фактический FWER ниже номинального, а мощность каждого отдельного теста снижается. Альтернативные методы (например, процедура Хохберга) могли бы дать более мощное, но менее консервативное решение; в настоящей работе мы намеренно выбрали консервативный вариант.

Совпадение тренировочного и тестового распределений для `train_real_250`. И `benchmark-real`, и `train_real_250` сформированы из одного и того же пула 500 реальных запросов (раздел 5.2.2). При отсутствии пересечений на уровне индивидуальных запросов (что проверено) распределения признаков всё равно близки. Поэтому высокие метрики `train_real_250` на `benchmark-real` отражают не только качество дообучения, но и распределенческое совпадение; выводы о превосходстве реальных данных над синтетическими следует формулировать именно в таком виде.

8. Программная реализация

Для проведения экспериментов по исследованию влияния дообучения большой языковой модели на качество классификации инструментов был разработан программный комплекс `ml_benchmark`. Система реализована на языке Python 3.11 и обеспечивает полный цикл экспериментальной работы: от генерации синтетических данных до статистического анализа результатов.

8.1. Архитектура системы

Программный комплекс построен по принципу модульной архитектуры с чётким разделением ответственности между слоями. Верхний уровень образует интерактивный CLI-интерфейс, реализованный с помощью библиотеки Rich и предоставляющий меню из 11 команд, каждая из которых соответствует отдельному этапу экспериментального процесса. Средний уровень составляет бизнес-логика, сосредоточенная в модуле `core/`: генерация данных, инференс моделей, вычисление метрик, статистический анализ, визуализация и формирование отчётов. Нижний уровень включает подсистемы хранения (`storage/`), конфигурации (`config/`) и LLM-клиентов (`clients/`).

Взаимодействие между слоями организовано через паттерн внедрения зависимостей (Dependency Injection). Главный цикл приложения, реализованный в классе `App` (модуль `cli.py`), при инициализации создаёт четыре ключевых объекта: `settings` (конфигурация из YAML-файлов и переменных окружения), `artifacts` (менеджер файловых артефактов), `registry` (реестр моделей) и `saves` (менеджер сохранений). Каждый обработчик команды получает эти объекты в качестве параметров, что обеспечивает слабую связанность компонентов и упрощает тестирование.

Для строгой типизации всех структур данных используются модели Pydantic, определённые в пакете `models/`. Они описывают схемы датасетов, экспериментов, метрик, отчётов групповой оценки и итоговых отчётов. Валидация данных на уровне моделей позволяет обнаруживать

ошибки на ранних стадиях и гарантирует целостность данных при их передаче между компонентами.

Подсистема LLM-клиентов реализована по принципу плагинной архитектуры. Абстрактный базовый класс `BaseClient` определяет интерфейс взаимодействия с языковыми моделями, включая типы `Message` и `ChatResponse`. Конкретные реализации, такие как `YandexCloudClient`, наследуют этот интерфейс и инкапсулируют специфику работы с API конкретных провайдеров. Такой подход позволяет подключать новых провайдеров без модификации существующего кода.

8.2. Подсистема статистического анализа

Центральным компонентом подсистемы статистического анализа является класс `GroupEvaluator`, реализованный в модуле `core/group_evaluator.py`. Данный класс обеспечивает статистически корректное сравнение нескольких моделей на одном и том же наборе тестовых примеров. Алгоритм его работы включает пять основных этапов.

На первом этапе выполняется построение бинарной матрицы результатов размерности $n_{\text{samples}} \times n_{\text{models}}$, где каждый элемент $b_{ij} \in \{0, 1\}$ отражает корректность предсказания j-й модели на i-м примере. Классификация предсказания как верного (TP, TN) или ошибочного (FP, FN) осуществляется модулем `Evaluator`, который выполняет парсинг вызовов инструментов из XML-ответов модели и сопоставляет их с эталонными значениями.

На втором этапе для каждой модели вычисляются доверительные интервалы метрик с помощью бутстрапа. Используется $B = 1000$ итераций ресемплирования с возвращением, что обеспечивает непараметрическую оценку распределения метрик без предположений о их аналитическом виде.

На третьем этапе выполняется проверка глобальной гипотезы H_0 с помощью критерия Кохрена Q (раздел 4.4.1, реализация из библиотеки `statsmodels`).

Четвёртый этап выполняется только при отклонении глобальной гипотезы и включает попарные сравнения моделей с помощью критерия Макнемара с поправкой Бонферрони (раздел 4.4.2).

На пятом этапе проводится анализ ошибок: для каждой модели формируется перечень примеров с ошибочной классификацией, что позволяет выявить систематические паттерны ошибок и понять, на каких типах запросов модель испытывает затруднения.

8.3. Подсистема инференса

Подсистема инференса реализована в классе `InferenceRunner` (модуль `core/runner.py`) и обеспечивает запуск языковых моделей на наборе тестовых примеров с учётом ограничений внешних API.

Для обеспечения устойчивости к сбоям реализован механизм чекпоинтов: после обработки каждого примера промежуточные результаты сохраняются на диск. При повторном запуске система автоматически обнаруживает последний чекпоинт и продолжает выполнение с того примера, на котором произошло прерывание. Это критически важно при работе с большими датасетами, поскольку один полный прогон может занимать значительное время.

Для соблюдения ограничений частоты запросов (rate limits) внешних API между последовательными вызовами модели выдерживается пауза в 0.5 секунды. Кроме того, реализована стратегия повторных попыток (retry) при получении временных ошибок от API, что повышает надёжность сбора данных.

Парсинг ответов модели выполняется классом `ToolCallParser` (модуль `core/evaluator.py`), который извлекает вызовы инструментов из XML-разметки ответа. Извлечённые вызовы сопоставляются с эталонными, после чего класс `MetricsCollector` подсчитывает количество истинно положительных (TP), ложноположительных (FP), истинно отрицательных (TN) и ложноотрицательных (FN) результатов и на их основе вычисляет итоговые метрики классификации.

8.4. Подсистема генерации данных

Генерация синтетических тестовых данных выполняется двумя компонентами. Класс `Generator` (модуль `core/generator.py`) формирует синтетические пользовательские запросы с помощью вызовов LLM, обеспечивая контроль баланса классов в итоговом датасете. Класс `Humanizer` (модуль `core/humanizer.py`) выполняет «очеловечивание» сгенерированных запросов — повторно обрабатывает синтетические примеры через LLM, чтобы придать им стилистическое разнообразие, характерное для реальных пользовательских формулировок. Полученные данные конвертируются в формат JSONL классом `DatasetConverter` для последующего использования при дообучении моделей.

8.5. Обеспечение качества кода

Качество программного комплекса поддерживается комплексом автоматизированных инструментов. Юнит-тестирование осуществляется с помощью фреймворка `pytest`, основная логика пайплайна покрыта unit-тестами. Статический анализ кода выполняется линтером `ruff`, а проверка типов — инструментом `mypy`. Совместное использование статической типизации `mypy` и валидации данных через Pydantic-модели во время исполнения обеспечивает двухуровневый контроль корректности: ошибки несовместимости типов обнаруживаются как на этапе разработки, так и при передаче данных между компонентами в процессе выполнения.

8.6. Технологический стек

Перечень основных библиотек и инструментов, используемых в проекте, приведён в таблице 15.

Таблица 15: Технологический стек проекта

Библиотека	**Назначение**
Python 3.11	Язык реализации
Pydantic	Описание и валидация схем данных
NumPy	Числовые операции, работа с матрицами результатов
statsmodels	Критерий Кохрена Q, критерий Макнемара
matplotlib	Построение графиков и визуализаций
Rich	Интерактивный CLI-интерфейс
PyYAML	Чтение конфигурационных файлов
pytest	Юнит-тестирование
ruff	Линтинг и форматирование кода
mypy	Статическая проверка типов

9. Заключение

В рамках настоящей работы проведено экспериментальное исследование влияния дообучения большой языковой модели YandexGPT 5.1 методом LoRA [19] на качество классификации инструментов в AI-агенте, функционирующем в домене недвижимости. Исследование выполнено на двух независимых бенчмарках общим объёмом 750 примеров (500 смешанных и 250 реальных) и охватывает 12 моделей с вариацией типа и объёма обучающих данных.

Основные задачи, поставленные в начале исследования, были решены в полном объёме:

1. Формализована задача классификации инструментов как задача бинарной классификации с классами GEO (`ResolveEntities`) и TAG (`SearchTags`); сформированы два независимых бенчмарка для раздельной и совместной оценки моделей.

2. Разработан модульный программный пайплайн, обеспечивающий полный цикл — от подготовки обучающих данных и запуска дообучения через API YandexGPT до автоматизированного inference и вычисления метрик с визуализацией результатов.

3. Подготовлено 12 конфигураций моделей (1 базовая и 11 дообученных) с систематической вариацией источника данных (синтетические, реальные, смешанные), классового баланса и объёма выборки (от 50 до $\sim$1500 примеров), и проведено 24 цикла оценки на двух бенчмарках.

4. Выполнен строгий статистический анализ результатов с применением критерия Кохрена Q для проверки глобальной гетерогенности и попарного критерия Макнемара с поправкой Бонферрони для множественных сравнений.

Главные научные результаты работы заключаются в следующем:

- **Поддержана рабочая гипотеза о стилистической природе выбора инструмента.** Дообучение методом LoRA повышает

accuracy с 81,8% до 99,4%, а Macro F_1 — с 0,526 до 0,996. Различия статистически значимы (раздел 6.7). Результат согласуется с гипотезой поверхностного выравнивания [16]: небольшой объём примеров корректного формата достаточен для радикального улучшения качества. Это эмпирическая поддержка аналогии, а не строгое доказательство самой SAH применительно к tool selection — вопрос о том, действительно ли дообучение затрагивает лишь «форму», а не «знания» модели в этом контексте, выходит за рамки настоящей работы.

- **Установлена оптимальная стратегия формирования обучающей выборки.** Модель на смешанном датасете (250 реальных + 250 синтетических примеров) достигает наилучших результатов и демонстрирует минимальную деградацию при переходе на реальные данные. Увеличение объёма синтетических данных сверх 500 примеров приводит к ухудшению, что свидетельствует о переобучении на артефакты генерации. Качество обучающих данных оказывается важнее их объёма (раздел 6.5).

- **Выявлен эффект консервации ошибок.** 73,6% примеров бенчмарка классифицируются одинаково всеми моделями, что указывает на устойчивое ядро «лёгких» и «трудных» запросов. Ошибки воспроизводятся на обоих бенчмарках, что свидетельствует об их детерминированной природе — они обусловлены систематическими слабостями модели на определённых типах запросов, а не стохастикой генерации.

Практическая значимость работы определяется следующим. Во-первых, разработанный программный пайплайн (описанный в разделе 8 и предшествующей работе [33]) может быть использован для дообучения и систематической оценки моделей YandexGPT в произвольных задачах классификации. Во-вторых, сформулированы конкретные рекомендации по формированию обучающих данных: использование смешанных выборок (реальные + синтетические) предпочтительнее наращивания объёма чисто синтетических примеров, что позволяет

снизить затраты на ручную разметку без потери качества. В-третьих, показано, что метод LoRA обеспечивает переход от неприемлемого уровня качества к практически безошибочной классификации (раздел 6.2). Ограничения, накладываемые на интерпретацию полученных результатов, систематизированы в разделе 7.

9.1. Направления дальнейших исследований

Проведённое исследование открывает ряд перспективных направлений для продолжения работы:

- **Переход к multi-label классификации.** В текущей постановке каждый запрос относится ровно к одному классу. Однако на практике встречаются запросы, которые выглядят географически, но на самом деле описывают характеристики инфраструктуры — например, «рядом с парком», «у озера», «недалеко от леса». Такие запросы классифицируются как TAG, хотя содержат пространственные указания. Расширение модели до multi-label классификации позволит корректно обрабатывать подобные пограничные случаи.

- **Исследование альтернативных PEFT-методов.** В данной работе использован исключительно метод LoRA. Сравнение с другими подходами к параметрически эффективному дообучению — QLoRA, Prompt Tuning [18], Adapter Tuning — позволит определить, является ли LoRA оптимальным выбором для задачи tool selection.

- **Обобщение на другие LLM.** Все эксперименты проведены на семействе моделей YandexGPT 5.1. Проверка воспроизводимости результатов на моделях иных архитектур и провайдеров позволит установить, в какой мере полученные закономерности являются универсальными, а в какой — специфичными для конкретной модели.

Таким образом, настоящая работа вносит вклад в понимание механизмов адаптации больших языковых моделей к задаче выбора инструментов и демонстрирует, что дообучение методом LoRA на относительно небольшой, но качественно сформированной обучающей выборке является эффективной стратегией повышения качества классификации в AI-агентах.

Список литературы

[1] API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs / Minghao Li, Yingxiu Zhao, Bowen Yu et al. // arXiv preprint arXiv:2304.08244. — 2023.

[2] Attention Is All You Need / Ashish Vaswani, Noam Shazeer, Niki Parmar et al. // Advances in Neural Information Processing Systems. — Vol. 30. — 2017.

[3] Yan Fanjia, Mao Huanzhi, Ji Charlie Cheng-Jie et al. Berkeley Function Calling Leaderboard. — https://gorilla.cs.berkeley.edu/leaderboard. — 2024. — Дата обращения: 2026-04-01.

[4] Bickel P. J., Freedman D. A. Some Asymptotic Theory for the Bootstrap // The Annals of Statistics. — 1981. — Vol. 9, no. 6. — P. 1196–1217.

[5] Bonferroni Carlo. Teoria statistica delle classi e calcolo delle probabilità // Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commericiali di Firenze. — 1936. — Vol. 8. — P. 3–62.

[6] Cantelli F. P. Sulla determinazione empirica delle leggi di probabilità // Giornale dell'Istituto Italiano degli Attuari. — 1933. — Vol. 4. — P. 421–424.

[7] Cochran William G. The Comparison of Percentages in Matched Samples // Biometrika. — 1950. — Vol. 37, no. 3/4. — P. 256–266.

[8] Dietterich Thomas G. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms // Neural Computation. — 1998. — Vol. 10, no. 7. — P. 1895–1923.

[9] Dunn Olive Jean. Multiple Comparisons Among Means // Journal of the American Statistical Association. — 1961. — Vol. 56, no. 293. — P. 52–64.

[10] Efron Bradley. Bootstrap Methods: Another Look at the Jackknife // The Annals of Statistics. — 1979. — Vol. 7, no. 1. — P. 1–26.

[11] Efron B., Tibshirani R. J. An Introduction to the Bootstrap. — New York : Chapman & Hall, 1993.

[12] The False Promise of Imitating Proprietary LLMs / Arnav Gudibande, Eric Wallace, Charlie Snell et al. // arXiv preprint arXiv:2305.15717. — 2023.

[13] Glivenko V. Sulla determinazione empirica delle leggi di probabilità // Giornale dell'Istituto Italiano degli Attuari. — 1933. — Vol. 4. — P. 92–99.

[14] Gorilla: Large Language Model Connected with Massive APIs / Shishir G Patil, Tianjun Zhang, Xin Wang, Joseph E Gonzalez // arXiv preprint arXiv:2305.15334. — 2023.

[15] Holm Sture. A Simple Sequentially Rejective Multiple Test Procedure // Scandinavian Journal of Statistics. — 1979. — Vol. 6, no. 2. — P. 65–70.

[16] LIMA: Less Is More for Alignment / Chunting Zhou, Pengfei Liu, Puxin Xu et al. // Advances in Neural Information Processing Systems. — Vol. 36. — 2023. — P. 55006–55021.

[17] Language Models are Unsupervised Multitask Learners / Alec Radford, Jeffrey Wu, Rewon Child et al. // OpenAI blog. — 2019. — Vol. 1, no. 8. — P. 9.

[18] Lester Brian, Al-Rfou Rami, Constant Noah. The Power of Scale for Parameter-Efficient Prompt Tuning // arXiv preprint arXiv:2104.08691. — 2021.

[19] LoRA: Low-Rank Adaptation of Large Language Models / Edward J Hu, Yelong Shen, Phillip Wallis et al. // arXiv preprint arXiv:2106.09685. — 2022.

[20] McNemar Quinn. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages // Psychometrika. — 1947. — Vol. 12, no. 2. — P. 153–157.

[21] Overcoming Catastrophic Forgetting in Neural Networks / James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz et al. // Proceedings of the National Academy of Sciences. — 2017. — Vol. 114, no. 13. — P. 3521–3526.

[22] Parameter-Efficient Fine-Tuning of Large-Scale Pre-Trained Language Models / Ning Ding, Yujia Qin, Guang Yang et al. // Nature Machine Intelligence. — 2023. — Vol. 5. — P. 220–235.

[23] Parameter-Efficient Transfer Learning for NLP / Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski et al. // International Conference on Machine Learning / PMLR. — 2019. — P. 2790–2799.

[24] ReAct: Synergizing Reasoning and Acting in Language Models / Shunyu Yao, Jeffrey Zhao, Dian Yu et al. // arXiv preprint arXiv:2210.03629. — 2023.

[25] The Rise and Potential of Large Language Model Based Agents: A Survey / Zhiheng Xi, Wenxiang Chen, Xin Guo et al. // arXiv preprint arXiv:2309.07864. — 2023.

[26] Self-Instruct: Aligning Language Models with Self-Generated Instructions / Yizhong Wang, Yeganeh Kordi, Swaroop Mishra et al. // arXiv preprint arXiv:2212.10560. — 2023.

[27] Sokolova Marina, Lapalme Guy. A Systematic Analysis of Performance Measures for Classification Tasks // Information Processing & Management. — 2009. — Vol. 45, no. 4. — P. 427–437.

[28] Tool Learning with Foundation Models / Yujia Qin, Shengding Liang, Yining Ye et al. // arXiv preprint arXiv:2304.08354. — 2024.

[29] ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs / Yujia Qin, Shihao Liang, Yining Ye et al. // arXiv preprint arXiv:2307.16789. — 2023.

[30] Toolformer: Language Models Can Teach Themselves to Use Tools / Timo Schick, Jane Dwivedi-Yu, Roberto Dessì et al. // arXiv preprint arXiv:2302.04761. — 2023.

[31] Training Language Models to Follow Instructions with Human Feedback / Long Ouyang, Jeffrey Wu, Xu Jiang et al. // Advances in Neural Information Processing Systems. — 2022. — Vol. 35. — P. 27730–27744.

[32] Yandex Cloud. YandexGPT API: Документация. — https://yandex.cloud/ru/docs/foundation-models/. — 2026. — Дата обращения: 2026-04-01.

[33] Горошко А. В. Исследование влияния дообучения большой языковой модели на качество классификации инструментов (Отчёт по учебной практике). — 2025. — СПбГУ, кафедра математики и компьютерных наук. Неопубликованная работа.